

Department of Information Technology

University of Gujrat

TextTalk

Session: BS-IT 2021-2025

Project Advisor: Mr. Muhammad Attique

Submitted By:

Abdullah	21014156-027
Rauf Ahmad	21014156-035
Umar Sheraz	21014156-036

Department of Information Technology
University of Gujrat

UNIVERSITY OF GUJRAT
STATEMENT OF SUBMISSION

This is to certify that **Abdullah** Roll No. **21014156-027**, **Rauf Ahmad** Roll No. **21014156-035** and **Umar Sheraz** Roll No. **21014156-036** have successfully completed the final project named as: **Genetics Insight**, at the Faculty of C & IT, University of Gujrat, to fulfill the partial requirement of the degree of **BS in Information Technology**.

Project Management Office
Department of Information Technology
Faculty of C & IT

Project Advisor
Designation
Dept. of Information Technology

Head of Department
Dept. of Information Technology
HH Campus UOG

Acknowledgement

We truly acknowledge the co-operation and help make by **Sir Fiaz Majeed, HOD.IT Department, University of Gujrat**. He has been a constant source of guidance throughout the course of this project. We would also like to thank **Mr. Muhammad Attique** for his help and guidance throughout this project. We are also thankful to our friends and families whose silent support led us to complete our project.

1- Abdullah

2- Rauf Ahmad

3- Umar Sheraz

Date: January 30, 2025

Abstract

TextTalk is an AI-powered web application designed to transform how users interact with PDF documents. By leveraging advanced natural language processing through **LangChain.js** and large language models (LLMs) like **GPT**, TextTalk enables users to upload, query, and manipulate PDF content in real-time. The platform provides features such as AI-driven document analysis, paraphrasing, and section extraction, making document navigation intuitive and efficient.

The system architecture integrates **Next.js** for seamless frontend and backend operations, **Tailwind CSS** for a responsive user interface, **and Clerk** for secure user authentication, and **Prisma with MongoDB** for scalable data management. An **in-memory vector database** ensures lightning-fast access to document embeddings, enhancing user experience.

TextTalk addresses the limitations of traditional static PDF viewers by offering intelligent and interactive document engagement, making it a valuable tool for students, researchers, and professionals. With its free, user-friendly approach, TextTalk redefines document management through the power of AI.

TABLE OF CONTENTS

CHAPTER# 01 PROJECT FEASIBILITY REPORT	7
--	---

1.1. INTRODUCTION.....	8
1.2. PROJECT/PRODUCT FEASIBILITY REPORT.....	8
1.2.1. Technical Feasibility.....	8
1.2.2. Operational Feasibility.....	9
1.2.3. Economic Feasibility	9
1.2.4. Schedule Feasibility.....	9
1.2.5. Specification Feasibility.....	9
1.2.6. Information Feasibility	9
1.2.7. Motivational Feasibility.....	10
1.2.8. Legal & Ethical Feasibility.....	10
1.3. PROJECT/PRODUCT SCOPE	10
1.4. PROJECT/PRODUCT COSTING	10
Unadjusted Function Points (UFP)	10
Adjusted Function Points (AFP):.....	11
1.6. INTRODUCTION TO TEAM MEMBER AND THEIR SKILL SET	16
1.7. TASK AND MEMBER ASSIGNMENT TABLE.....	16
1.10. TOOLS AND TECHNOLOGY WITH REASONING	16
1.11. VISION DOCUMENT	17
1.12. RISK LIST	19
1.13. PRODUCT FEATURES/ PRODUCT DECOMPOSITION.....	20
CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION(FOR OBJECT ORIENTED APPROACH)	22
2.1 INTRODUCTION:	23
3.1.2. Identifying External Entities	24
2.1.3. Context Level Data Flow Diagram:	25
2.1.4. Capture "shall" Statements:.....	26
2.1.5. Allocate Requirements:.....	26
2.1.6. Prioritize Requirements:.....	27
2.2.10. High Level Use case Diagram:.....	28
2.2. 11. Analysis Level Use case Diagram:	29
CHAPTER 3: DESIGN DOCUMENT (FOR OBJECT ORIENTED APPROACH)31	
3.1. INTRODUCTION:	32
3.2. DOMAIN MODEL	33
3.3. SYSTEM SEQUENCE DIAGRAM	34
3.5. COLLABORATION DIAGRAM.....	34
3.6. OPERATION CONTRACTS	35
3.7. DESIGN CLASS DIAGRAM.....	37
3.9. DATA MODEL.....	39
CHAPTER 3: SOFTWARE REQUIREMENT SPECIFICATION AND DESIGN DOCUMENT (FOR STRUCTURED APPROACH)	40
3.1. INTRODUCTION:	41
3.2. ENTITY RELATIONSHIP DIAGRAM:	41
3.3. DATA FLOW DIAGRAM (FUNCTIONAL MODEL).....	44
3.4. STATE TRANSITION DIAGRAM.....	45

3.5. ARCHITECTURAL DESIGN	47
3.6. COMPONENT LEVEL DESIGN	48
CHAPTER 4: USER INTERFACE DESIGN	49
4.1. INTRODUCTION.....	50
4.2. SITE MAPS	50
4.3. STORY BOARDS	51
<i>Storyboard 1: User Login & Registration</i>	51
<i>Storyboard 2: Uploading and Chatting with PDF</i>	54
4.4. NAVIGATIONAL MAPS:	60
4.5 TRACE-ABILITY MATRIX.....	61
CHAPTER 5: SOFTWARE TESTING	62
5.1 INTRODUCTION:	63
5.2. TEST PLAN.....	63
5.3. TEST DESIGN SPECIFICATION	65
5.4. TEST CASE SPECIFICATION.....	67
5.5. TEST PROCEDURE SPECIFICATION	67
5.6. TEST ITEM TRANSMITTAL REPORT	68
5.6.1. <i>Purpose</i>	68
5.6.2. <i>Outline</i>	68
5.7. TEST LOG	69
5.7.1. <i>Purpose</i>	69
5.7.2. <i>Outline</i>	69
5.8. TEST INCIDENT REPORT	69
5.8.1. <i>Purpose</i>	69
5.8.2. <i>Outline</i>	69
5.9. TEST SUMMARY REPORT.....	70
5.9.1. <i>Purpose</i>	70
5.9.2. <i>Outline</i>	70

Chapter# 01 Project Feasibility Report

1.1. Introduction

TextTalk is an AI-driven web application designed to revolutionize document interaction by enabling users to chat with PDF content. Traditional document viewers are limited to static reading, which can be time-consuming and inefficient for extracting specific information. TextTalk overcomes this limitation by integrating advanced AI capabilities to provide meaningful responses to user queries, extract insights, and navigate document content seamlessly.

The application allows users to upload PDF files and interact with them in real-time through a chat interface powered by **LangChain.js** and large language models like **GPT**. With its innovative architecture built using **Next.js**, **Tailwind CSS**, and **MongoDB**, TextTalk ensures a scalable, secure, and user-friendly experience.

Whether for students seeking study material highlights, researchers exploring detailed reports, or professionals analyzing lengthy documents, TextTalk offers an intelligent, time-saving solution that enhances productivity and document engagement.

1.2. Project/Product Feasibility Report

When a project is started the first matter to establish is to assess the feasibility of a project. There are many types of feasibilities:

- Technical
- Operational
- Economic
- Schedule
- Specification
- Information
- Motivational
- Legal and Ethical

1.2.1. Technical Feasibility

TextTalk is technically feasible due to the use of well-established frameworks and technologies:

Next.js for server-side rendering, offering a scalable and performant platform for frontend and backend development.

LangChain.js and large language models (LLMs) such as GPT for advanced document interaction and AI responses.

Tailwind CSS for responsive and clean UI design.

MongoDB for flexible and scalable database storage.

In-Memory Vector Database for quick access to embeddings, ensuring efficient query processing.

1.2.2. Operational Feasibility

TextTalk is designed for diverse user groups, including students, researchers, and professionals.

Ease of Use: A simple, intuitive chat interface makes it accessible even to non-technical users.

Real-Time Interaction: AI-powered responses enhance document comprehension and productivity.

Secure User Authentication: Integration with Clerk ensures secure access.

1.2.3. Economic Feasibility

Development Costs: Open-source tools such as Next.js, Tailwind CSS, and LangChain.js help reduce development expenses.

Hosting: Cloud-based hosting services offer flexible pricing for both development and deployment stages.

Maintenance: Using scalable technologies minimizes long-term maintenance costs.

Free Version Focus: Since TextTalk is a free application, no user subscription charges are planned, which reduces payment processing infrastructure costs.

1.2.4. Schedule Feasibility

The development timeline for TextTalk spans two semesters (8 months total), starting from October 8, 2024, and concluding in July 2025. The project milestones have been carefully planned using a detailed Gantt chart.

1.2.5. Specification Feasibility

The technical specifications of TextTalk are achievable using advanced tools and technologies.

Frontend: Built with Next.js for server-side rendering and file-based routing.

Backend: Powered by LangChain.js for AI-driven natural language processing.

Database: Utilizes MongoDB and In-Memory Vector Databases for secure and efficient data storage.

Security: Authentication with Clerk ensures secure user access and management.

The selected tools align well with the project scope, ensuring a robust and scalable solution.

1.2.6. Information Feasibility

Data Handling: The system efficiently handles large amounts of text data from uploaded PDF documents.

Information Processing: LangChain enables efficient document vectorization and intelligent content processing.

User Insights: Provides meaningful AI-powered responses, summarization, and paraphrasing of document content.

Data Storage: MongoDB ensures efficient management and retrieval of user data and document embeddings.

1.2.7. Motivational Feasibility

User Engagement: The AI-powered interactive document chat feature encourages users to adopt the application.

Real-World Use Cases: Ideal for researchers, students, and professionals looking to enhance productivity.

Innovation Factor: A cutting-edge solution compared to traditional PDF viewers, giving users more value and functionality.

Developer Motivation: The use of modern, scalable, and interesting technologies keeps development motivated and focused.

1.2.8. Legal & Ethical Feasibility

Data Privacy: TextTalk ensures compliance with data protection regulations by storing only essential user data securely.

Content Ownership: Provides transparency by ensuring that AI-generated responses reference their original source within the document.

Ethical Use: The system does not store user-uploaded document content unnecessarily, ensuring user confidentiality.

Compliance: Follows ethical AI practices, ensuring unbiased and accurate responses from language models.

1.3. Project/Product Scope

TextTalk is an AI-driven web application designed to help users interact with PDF content through natural language queries. Users can upload PDFs, ask questions, and receive intelligent, traceable responses powered by advanced AI models like GPT. The system ensures secure user access through Clerk and delivers fast, real-time responses using LangChain.js for document vectorization. Built with Next.js and Tailwind CSS, TextTalk offers a responsive and user-friendly interface.

The application targets students, researchers, and professionals who need quick insights from extensive documents. Future enhancements may include voice-based queries, multi-document support, and advanced analytics. Currently, it excludes features like image or audio file support and offline functionality. The project focuses on providing a free, smart solution for efficient document interaction.

1.4. Project/Product Costing

1.4.1. Project Cost Estimation by Function Point Analysis

Unadjusted Function Points (UFP)

UFP is calculated based on the following components:

Function Type	Count	Weight (Simple)	Weight (Average)	Weight (Complex)	Total Points
External Inputs (EI)	8	3	-	-	24
External Outputs (EO)	5	-	4	-	20
External Inquiries (EQ)	3	-	4	-	12
Internal Logic Files (ILF)	6	-	-	10	60
External Interface Files (EIF)	2	-	-	7	14
Total UFP					130

Value Adjustment Factor (VAF):

VAF is calculated based on 14 general system characteristics rated from 0 (No Impact) to 5 (Strong Impact). Assume an average total impact score of 35.

$$\text{VAF} = 0.65 + (35/100)$$

$$\text{VAF} = 1.00$$

Adjusted Function Points (AFP):

$$\text{AFP} = \text{UFP} \times \text{VAF}$$

$$\text{AFP} = 130 \times 1.00 = 130$$

1.4.2. Project Cost Estimation by using COCOMO'81 (Constructive Cost Model)

The **COCOMO'81** (Constructive Cost Model) is a software cost estimation model developed by Barry Boehm that helps in estimating the cost, effort, and schedule for a software project based on its size and complexity. It is a widely used method for project cost estimation.

COCOMO'81 has three models:

1. Basic Model

2. **Intermediate Model**
3. **Detailed Model**

In this case, we will use the **Detailed Model** to estimate the cost for **TextTalk**. This model considers various factors, such as product, hardware, personnel, and project attributes.

Steps to estimate cost using COCOMO'81 Detailed Model:

Step 1: Estimate the size of the software in KLOC (Kilo Lines of Code).

Based on your project (TextTalk), you can estimate the number of lines of code (LOC) expected. For estimation, let's assume the following based on the complexity of the project.

Let's assume:

- **Estimated Lines of Code (LOC) = 20,000** lines of code (based on the complexity of modules, APIs, etc.)

Step 2: Determine the effort multiplier (EAF).

COCOMO considers various attributes and assigns multipliers based on the characteristics of the project. These attributes include product complexity, team experience, tool support, etc.

Here are some typical COCOMO'81 attributes (with their ratings):

1. **Product reliability (RELY):** 1.15
2. **Database size (DATA):** 1.12
3. **Product complexity (CPLX):** 1.20
4. **Execution time constraint (TIME):** 1.05
5. **Storage constraint (STOR):** 1.10
6. **Virtual machine volatility (VIRT):** 1.00
7. **Required development schedule (SCED):** 1.10
8. **Programmer experience (PERS):** 1.25 (Assumed team members have a high level of experience)

To compute the **Effort Adjustment Factor (EAF)**, sum the values for all the factors:

$$\text{EAF} = 1.15 \times 1.12 \times 1.20 \times 1.05 \times 1.10 \times 1.00 \times 1.10 \times 1.25 = 1.97$$

Step 3: Estimate effort using the formula:

For the **Detailed COCOMO'81 model**, the effort (person-months) is given by:

$$\text{Effort} = 2.94 \times (\text{KLOC})^{1.10} \times \text{EAF}$$

Where:

- **KLOC** = 20 (as we assumed earlier)
- **EM** = 1.97

$$\text{Effort} = 2.94 \times (20)^{1.10} \times 1.97$$

$$\text{Effort} = 2.94 \times 26.49 \times 1.97 = 151.62 \text{ person-months}$$

Step 4: Estimate Development Time (in months):

COCOMO'81 provides a formula to estimate development time:

$$\text{Development Time} = 3.67 \times (\text{Effort})^{0.33}$$

Substitute the effort value (151.62):

$$\text{Development Time} = 3.67 \times (151.62)^{0.33} = 3.67 \times 11.34 = 41.6 \text{ months}$$

Step 5: Estimate the Cost:

The cost can be estimated by multiplying the total effort (person-months) by the hourly wage of the development team. Let's assume:

- **Average hourly wage** = \$40 per hour
- **Working hours per month** = 160 hours (assuming full-time work)

The total cost is:

$$\text{Cost} = \text{Effort (in person-months)} \times \text{Monthly hours} \times \text{Hourly wage}$$

$$\text{Cost} = 151.62 \times 160 \times 40 = 968,640 \text{ dollar}$$

1.4.3 Task Dependency Table

Task ID	Task Name	Predecessors	Duration (Weeks)
T1	Requirement Gathering	-	2
T2	System Design	T1	3
T3	Frontend Development	T2	4
T4	Backend Development	T2	5
T5	Database Integration	T4	3
T6	AI Module Integration	T3,T4,T5	4
T7	Testing & Debugging	T6	3
T8	Deployment	T7	2
T9	Documentation	T7	2
T10	Final Project Presentation	T8,T9	1

1.4.4 CPM - Critical Path Method

Network Diagram for the above-mentioned activities

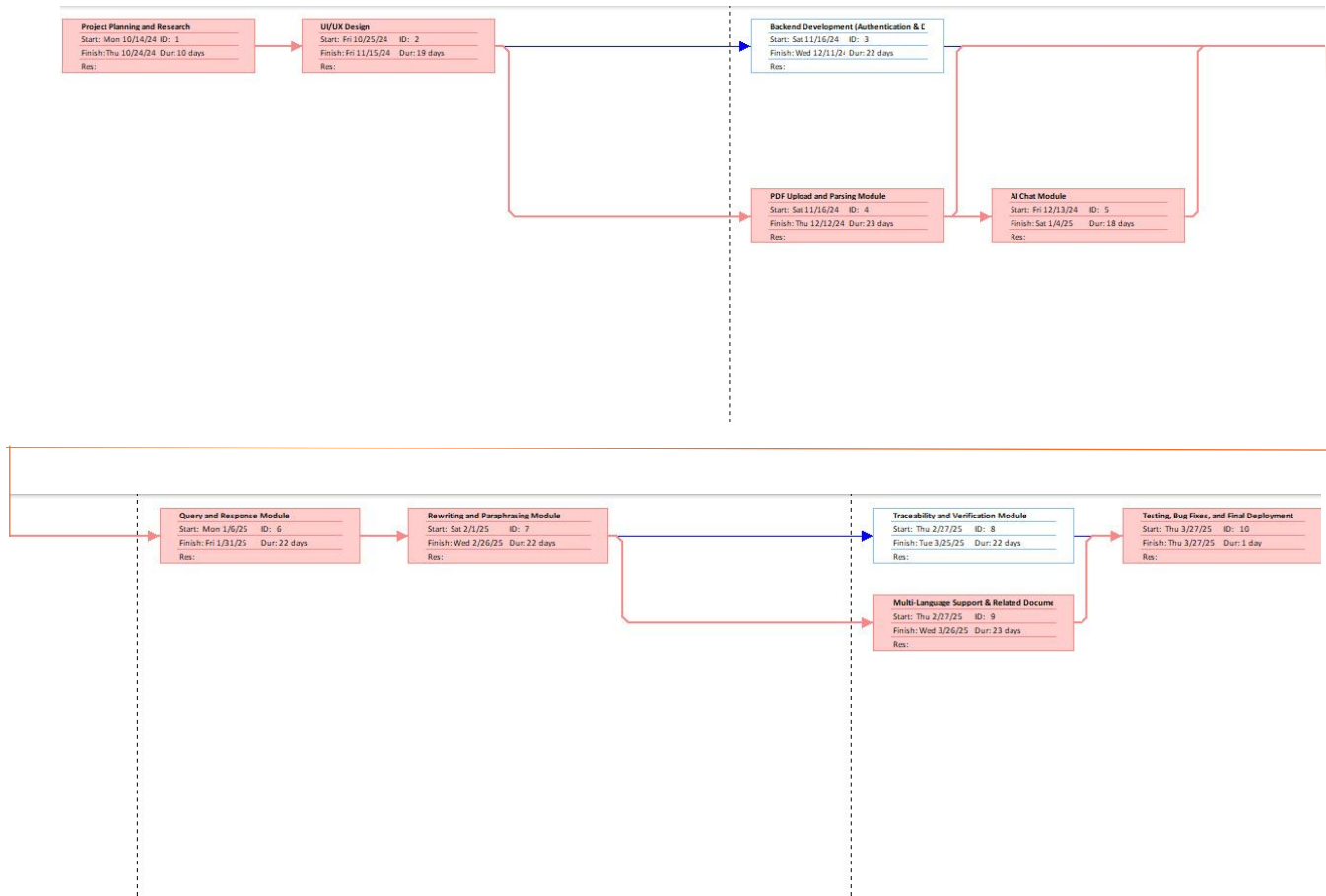


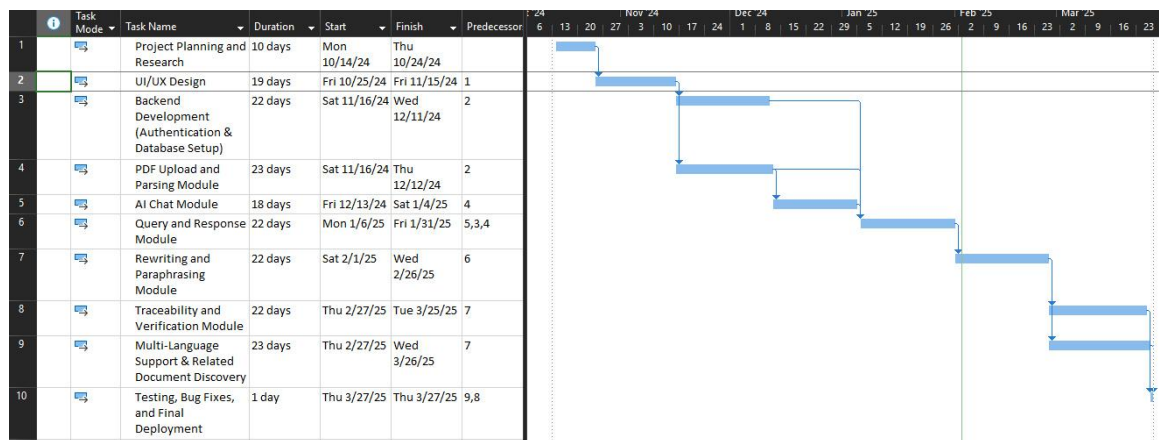
Table 1.4.5: Critical path

Activity	Description	Duration (Weeks)	Predecessor(s)	Earliest Start (ES)	Earliest Finish (EF)	Latest Start (LS)	Latest Finish (LF)	Slack (LF-EF)
A	Requirement Analysis	2	-	0	2	0	2	0
B	System Design	3	A	2	5	2	5	0
C	User Authentication Module	4	B	5	9	5	9	0

D	PDF Upload and Parsing	3	B	5	8	8	11	3
E	AI Chat Interface Module	5	C	9	14	9	14	0
F	Database Integration Module	4	D	8	12	11	15	3
G	Integration and Testing	6	E,F	14	20	14	20	0
H	UI/UX Refinement	3	G	20	23	20	23	0
I	Performance Optimization	2	H	23	25	23	25	0
J	Deployment	2	I	25	27	25	27	0
K	Documentation	2	I	25	27	25	27	0
L	Final Review and Delivery	1	J,K	27	28	27	28	0

Critical Path: A → B → C → E → G → H → I → J → L
Total Duration: 28 weeks

1.5 Gantt Chart:



1.6. Introduction to Team member and their skill set

Member name	Skills	Tasks
Abdullah	Application Designing ,Coding, Implementation and documentation	Designing Application Components, coding, Module, Testing, Database Coding and Designing
Umar Sheraz	Project Testing, Documentation, Planning	Application Designing and its Components, Modules , Structure, and Layout, Implementation
Rauf Ahmed	project Testing Documentation, Planning	Project Testing, Implementation, Documentation testing, Updating Module

1.7. Task and Member Assignment Table

Task	Assigned Member(s)	Duration (Weeks)	Dependencies
Requirements Gathering	All Members	2	None
Application Design	Abdullah,Umar Sheraz	3	Requirements Gathering
Database Design	Abdullah	2	Application Design
Module Development	Abdullah	5	Database Design
Frontend Component Design	Abdullah, Umar Sheraz	4	Application Design
Backend Integration	Abdullah	4	Module Development
Application Testing	Rauf Ahmed, Umar Sheraz	3	Backend Integration, UI Design
Documentation	Rauf Ahmed	2	Application Testing
Final Review and Deployment	All Members	2	Documentation

1.10. Tools and Technology with reasoning

- **Next.js**

Reasoning: Next.js offers server-side rendering (SSR) and static site generation (SSG), which enhance application speed and SEO. Its file-based routing and built-

in API routes simplify both frontend and backend development, making it ideal for scalable applications like TextTalk.

- **Tailwind CSS**

Reasoning: Tailwind CSS is a utility-first CSS framework that allows rapid development of a clean and responsive user interface without writing custom styles. This helps maintain a visually appealing and structured UI for the chat interface.

- **LangChain.js**

Reasoning: LangChain.js makes working with large language models (LLMs) like OpenAI's GPT efficient. It is used for document vectorization, enabling efficient AI-driven responses based on PDF content, which is crucial for TextTalk.

- **TypeScript**

Reasoning: TypeScript adds static typing to JavaScript, which helps detect errors early and improves maintainability. This is essential for managing the complex logic of multiple modules in TextTalk.

- **Clerk (Authentication)**

Reasoning: Clerk provides secure user authentication, supporting social logins and multi-factor authentication. It simplifies user session management, making the application more secure.

- **Prisma (ORM)**

Reasoning: Prisma is a type-safe Object-Relational Mapping (ORM) tool. It simplifies database queries and provides consistency when interacting with MongoDB, ensuring reliable data management.

- **MongoDB (User Database)**

Reasoning: MongoDB, being a NoSQL database, offers flexibility in storing unstructured data. It is chosen for its scalability and performance in securely storing user information.

- **In-Memory Vector Database**

Reasoning: This database stores document embeddings generated by LangChain.js, enabling fast and efficient querying for AI-based responses. The choice prioritizes performance for real-time interactions.

1.11. Vision Document

1. Introduction

TextTalk is an AI-powered web application designed to help users interact with and gain insights from their PDF documents. The system leverages advanced AI technologies to provide meaningful responses, paraphrasing, and document content analysis, creating a seamless experience for individuals and organizations managing document-intensive workflows.

2. Business Goals

- Deliver a free AI-driven document assistant that enhances user productivity.
- Simplify complex document analysis through conversational AI.

- Ensure data security and privacy for uploaded documents.
- Offer a user-friendly interface to maximize accessibility.

3. Stakeholders

- **Primary Users:** Students, researchers, and professionals who need to interact with PDF content.
- **Development Team:** Software developers, UI/UX designers, and AI specialists.
- **Potential Organizations:** Educational institutions and companies handling large document volumes.

4. Problem Statement

Managing large PDF documents is often time-consuming and inefficient. Users face difficulties in searching for specific information and summarizing content. TextTalk addresses these challenges by providing an intelligent solution for document comprehension and query resolution.

5. Solution Overview

TextTalk leverages AI-powered tools and machine learning models to allow users to chat with their PDF content. By simply uploading a document, users can:

- Extract key information through natural language queries.
- Paraphrase complex content for better understanding.
- Navigate efficiently to relevant sections based on their questions.

6. Key Features

- **PDF Upload and Parsing:** Easy document upload with intelligent text extraction.
- **AI Chat Interface:** Real-time interaction with the document content using AI.
- **Rewriting and Paraphrasing:** Simplifies complex text for better comprehension.
- **Multi-Language Support:** Allows querying and response in different languages.
- **Related Document Discovery:** Suggests relevant sections or documents based on queries.

7. Target Market

- **Educational Sector:** Students and academic researchers.
- **Business Professionals:** Individuals handling reports and documentation.
- **Freelancers:** Writers, analysts, and editors working with extensive document content.

8. Project Scope

- **Initial Release:** Free version with essential functionalities for document interaction.

- **Future Enhancements:** Support for additional languages and premium features like advanced document analytics.

9. Technology Stack

- **Frontend:** Next.js and Tailwind CSS for responsive design.
- **Backend:** LangChain.js for AI interactions and Clerk for user authentication.
- **Database:** Prisma with MongoDB for data management and an in-memory vector database for fast document query processing.

10. Success Metrics

- Increased user engagement and satisfaction.
- Faster document comprehension and response time.
- Positive user feedback and adoption across different sectors.

1.12. Risk List

1. Technical Risks

- **Integration Issues:** Challenges in integrating LangChain.js with multiple LLMs and vector databases.
- **Performance Bottlenecks:** High response time for large PDF files during vectorization and AI queries.
- **Data Storage Problems:** In-memory storage limitations affecting scalability.

2. Security Risks

- **User Data Privacy:** Risk of unauthorized access to user data and uploaded documents.
- **Authentication Vulnerabilities:** Potential weaknesses in multi-factor and social login mechanisms.

3. Development Risks

- **Time Overruns:** Delays in completing key modules like AI chat and document parsing due to technical complexities.
- **Resource Allocation:** Limited availability of skilled developers for AI and backend integration.

4. Operational Risks

- **System Downtime:** Unexpected server failures affecting user experience.
- **Maintenance Complexity:** Difficulty in managing updates for AI models and database systems.

5. Market Risks

- **User Adoption:** Low user interest due to competition with similar AI-driven applications.
- **AI Model Costs:** Rising expenses for accessing premium AI models like GPT or Gemini.

6. Legal and Ethical Risks

- **Copyright Issues:** Legal concerns over users uploading copyrighted PDF documents.
- **AI Response Accountability:** Ethical concerns regarding the accuracy and accountability of AI-generated responses.

7. External Risks

- **Technology Dependency:** Dependency on third-party services (e.g., Clerk, LangChain, MongoDB).
- **Regulatory Changes:** Compliance issues with evolving data protection laws (like GDPR).

1.13. Product Features/ Product Decomposition

1. User Authentication Module

- Secure user registration and login.
- Social login support (Google, etc.).
- Multi-factor authentication for enhanced security.

2. PDF Upload and Parsing Module

- Easy drag-and-drop upload of PDF documents.
- Text extraction from PDF files for processing.
- Support for large document handling.

3. AI Chat Interface Module

- Conversational interaction with uploaded PDF content.
- Natural language query support for document analysis.
- AI-driven responses based on document content.

4. Document Vectorization and Storage Module

- Embedding of document content for efficient search and query processing.
- In-memory vector storage for fast query performance.

5. Query and Response Module

- AI-generated responses based on user questions.
- Section-based matching for relevant answers.

6. Rewriting and Paraphrasing Module

- Allows users to request simplified or rephrased content.
- Ensures the original meaning is preserved during paraphrasing.

7. Related Document Discovery Module

- Suggests relevant documents based on user queries.
- Assists in navigating large datasets efficiently.

8. Traceability and Verification Module

- AI-generated responses traceable to their original source within the PDF.
- Enhances transparency and user trust.

9. Multi-Language Support Module

- Enables interaction in various languages.
- Translation of both queries and responses for global accessibility.

10. Multi-LLM Support Module

- Integration with multiple language models (e.g., GPT, Gemini).
- Provides flexibility in choosing the best AI model for specific tasks.

Chapter 2: Software Requirement Specification (For Object Oriented Approach)

2.1 Introduction:

This chapter presents the **Software Requirement Specification (SRS)** for the project titled **TextTalk**. The SRS outlines all the functional and non-functional requirements, use case models, and system constraints based on the **Object-Oriented Development Approach**.

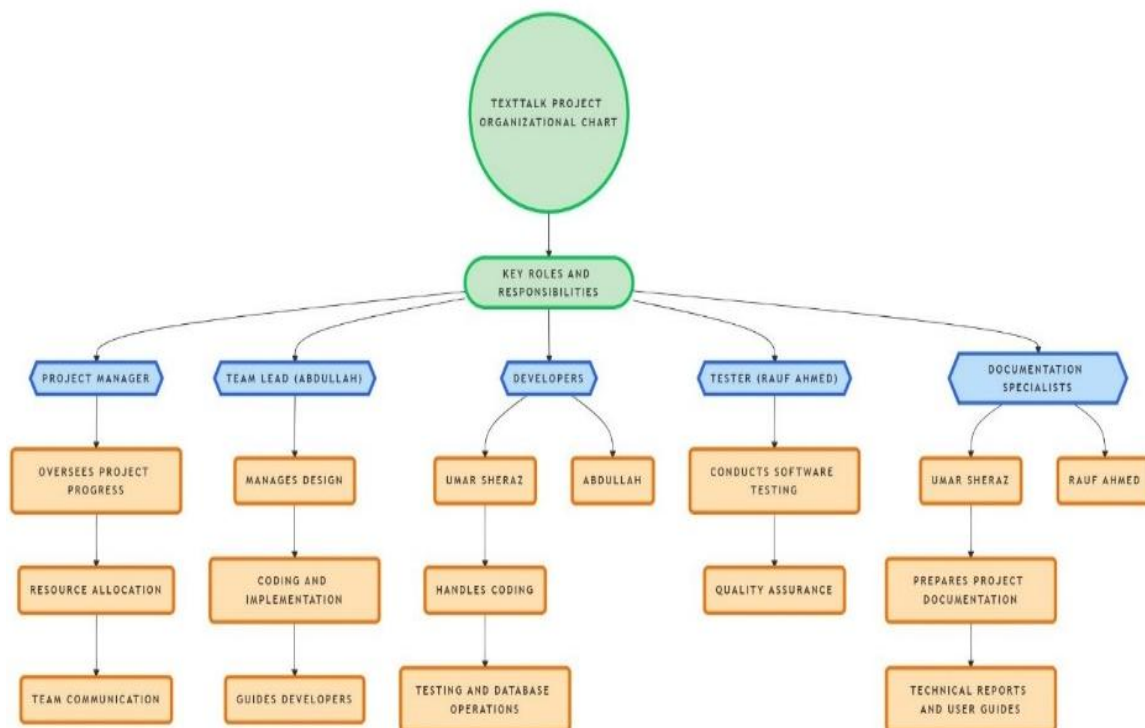
TextTalk is a web-based application that allows users to **upload PDF documents and interact with them using natural language**. Users can ask questions, paraphrase content, or translate sections with the help of AI models (such as GPT) and text embedding engines (LangChain.js). The system is designed to be user-friendly, responsive, and scalable.

This document serves as a guide for the development, testing, and deployment teams. It ensures all stakeholders have a **common understanding of system functionality and behavior**.

Existing System:

Current systems for interacting with document content are either limited to static searches or require manual reading and interpretation. Traditional applications do not support intelligent document-based conversations or AI-driven insights. Moreover, they lack features like document vectorization and real-time chat interfaces for efficient querying.

Organizational Chart:



2.1.2. Identifying External Entities

□ **User:**

- Role: End users interacting with the system.
- Responsibilities: Upload PDFs, query document content, and receive AI-based responses.
- Types: Students, researchers, and professionals.

□ **Authentication Provider (Clerk):**

- Role: Provides secure user authentication and session management.
- Responsibilities: Handle user registration, login, and multi-factor authentication.

□ **LLM Providers (OpenAI, Gemini, etc.):**

- Role: Provide AI models for language processing.
- Responsibilities: Respond to user queries, summarize, and paraphrase document content.

□ **Database (MongoDB):**

- Role: Stores user data and document information.
- Responsibilities: Securely store and retrieve user-uploaded document metadata.

□ **In-Memory Vector Database:**

- Role: Fast access storage for document embeddings.
- Responsibilities: Improve query performance by storing vectorized document data.

b. Perform Refinement:

1. User Refinement:

- *User Actions:* Upload documents, interact through chat interface, and manage document history.
- *Dependencies:* Dependent on the AI model for accurate responses and document storage for history retrieval.

2. Authentication Provider Refinement:

- *Security Role:* Ensures secure access and session handling.
- *Failure Handling:* Logs users out during session errors or security breaches.

3. LLM Providers Refinement:

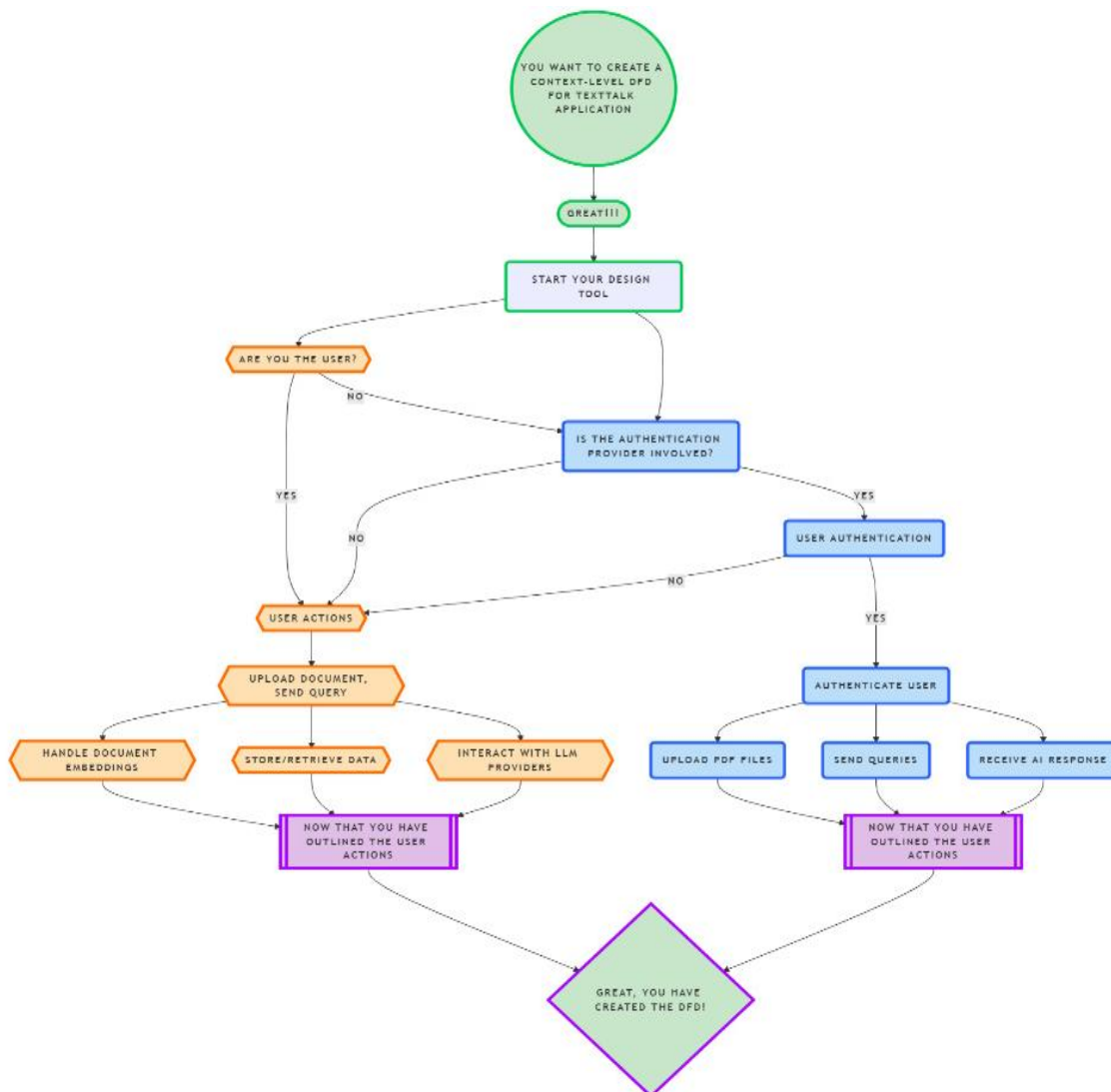
- *Function:* Process natural language queries and generate human-like responses.
- *Scalability:* Allows support for multiple language models based on user preferences.

4. Database Refinement:

- *Data Role*: Secure storage for user and document data.
 - *Optimization*: Indexing and efficient data retrieval for performance.
5. **Vector Database Refinement:**
- *Performance Role*: Enhances query speed through embedding storage.
 - *Data Handling*: Temporary storage for real-time vector operations.

2.1.3. Context Level Data Flow Diagram:

Context level data flow diagram contains only one process, representing the entire system. The process is given the number zero and all external entities are shown on the context diagram as well as major data flow to and from them. The diagram does not contain any data stores.



2.1.4. Capture "shall" Statements:

Requirement ID	Shall Statement
RQ-01	The system shall provide secure user authentication through Clerk, allowing social logins and multi-factor authentication.
RQ-02	Users shall be able to upload PDF documents for content extraction and interaction.
RQ-03	The system shall parse uploaded PDFs and generate document embeddings for efficient querying.
RQ-04	The AI model shall provide accurate and context-aware responses to user queries.
RQ-05	The application shall maintain a responsive and user-friendly interface built with Next.js and Tailwind CSS.
RQ-06	The system shall store user data securely in MongoDB.
RQ-07	The vector database shall handle and retrieve document embeddings for quick response generation.
RQ-08	The platform shall support multiple LLMs, allowing users to switch between GPT, Gemini, and other models.
RQ-09	The system shall ensure traceability of AI-generated responses to their original PDF content.
RQ-10	The application shall handle paraphrasing and rewriting requests while preserving the original document's meaning.
RQ-11	The system shall provide multilingual query and response support.
RQ-12	The platform shall display related document suggestions based on user queries.

2.1.5. Allocate Requirements:

Requirement ID	Requirement Description	Allocated Module
RQ-01	Secure user authentication with social login and multi-factor support.	User Authentication Module
RQ-02	Uploading of PDF documents for content extraction.	PDF Upload and Parsing Module
RQ-03	Parsing and embedding document content for efficient querying.	Document Vectorization Module
RQ-04	AI-based accurate and context-aware responses to queries.	AI Chat Interface Module
RQ-05	Responsive and user-friendly UI.	Frontend UI Module (Next.js & Tailwind CSS)
RQ-06	Secure storage of user data.	Database Module (MongoDB)

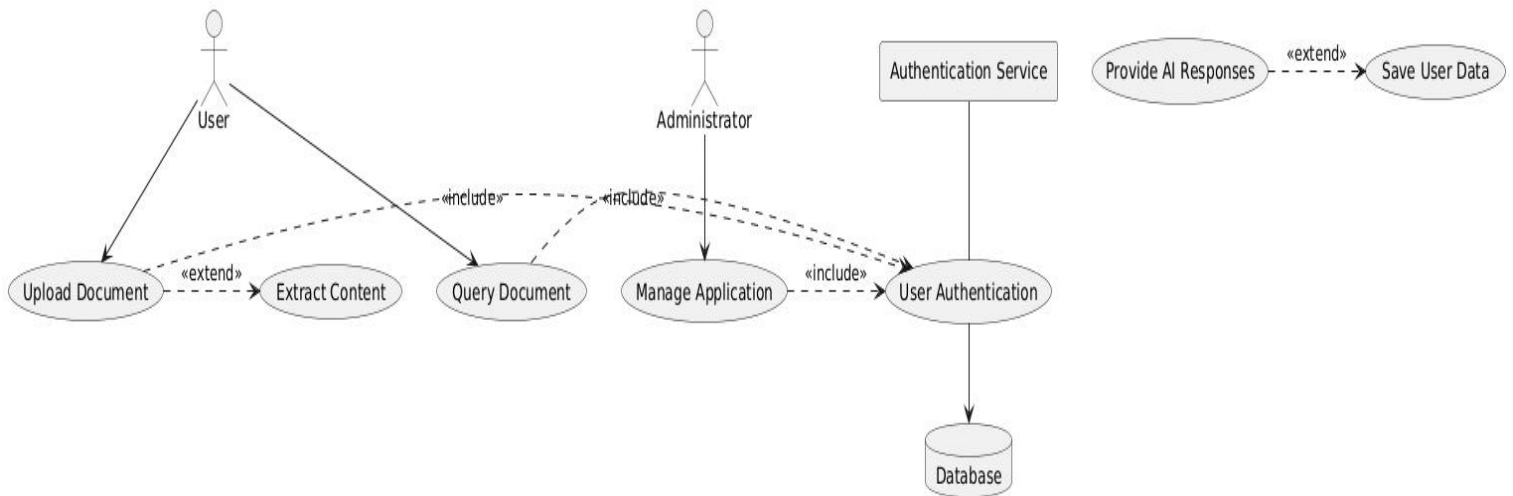
RQ-07	Handling and retrieving document embeddings quickly.	In-Memory Vector Database Module
RQ-08	Support for multiple LLMs.	Multi-LLM Support Module
RQ-09	Ensuring traceability of AI-generated responses.	Traceability and Verification Module
RQ-10	Paraphrasing and rewriting requests handling.	Rewriting and Paraphrasing Module
RQ-11	Multilingual query and response support.	Multi-Language Support Module
RQ-12	Display related document suggestions based on user queries.	Related Document Discovery Module

2.1.6. Prioritize Requirements:

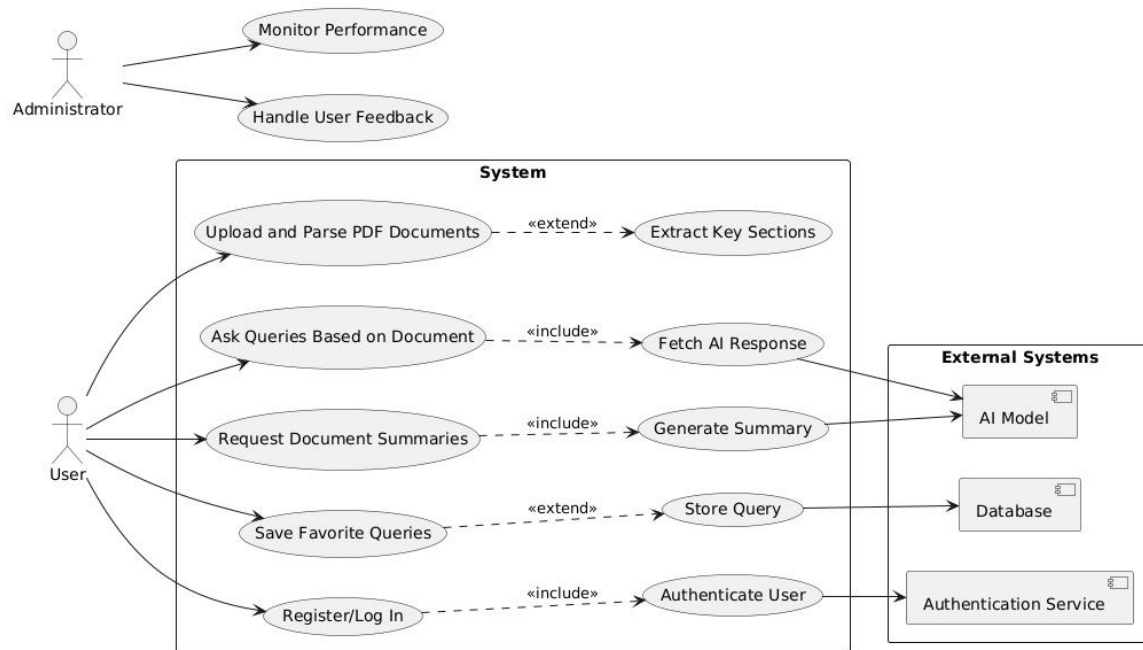
Requirement ID	Requirement Description	Priority (High/Medium/Low)	Reasoning
RQ-01	Secure user authentication with social login and multi-factor support.	High	Essential for user access and security.
RQ-02	Uploading of PDF documents for content extraction.	High	Core functionality for document-based interactions.
RQ-03	Parsing and embedding document content for efficient querying.	High	Necessary for AI-powered content search and responses.
RQ-04	AI-based accurate and context-aware responses to queries.	High	Key feature for user engagement and satisfaction.
RQ-05	Responsive and user-friendly UI.	Medium	Enhances user experience but not critical for core functionality.
RQ-06	Secure storage of user data.	High	Essential for maintaining user privacy and data integrity.
RQ-07	Handling and retrieving document embeddings quickly.	Medium	Improves performance for real-time interactions.

RQ-08	Support for multiple LLMs.	Low	Adds flexibility but not immediately essential.
RQ-09	Ensuring traceability of AI-generated responses.	Medium	Increases transparency and trustworthiness.
RQ-10	Paraphrasing and rewriting requests handling.	Low	Enhances usability but not a core requirement.
RQ-11	Multilingual query and response support.	Medium	Broadens user base but can be secondary.
RQ-12	Display related document suggestions based on user queries.	Low	Improves navigation but not critical for initial release.

2.2.10. High Level Use case Diagram:



2.2. 11. Analysis Level Use case Diagram:



2.2.12. Usecase Description

Use Case Name	Upload and Parse PDF
Description	This use case allows users to upload a PDF document. The system extracts and parses the document content for AI-based interactions.
Actors	User
Preconditions	The user must be logged into the system.
Postconditions	The PDF content is parsed and stored as embeddings for querying.
Main Flow	

- I. The user selects and uploads a PDF file.
- II. The system validates the file format.
- III. The system parses the document content.
- IV. The parsed content is vectorized and stored in the in-memory database.
- V. Confirmation is shown to the user.

Use Case Name	Ask Query Based on Document Content
---------------	-------------------------------------

Description	This use case allows users to ask questions based on uploaded PDF content and get AI-generated responses.
Actors	User
Preconditions	The user must have uploaded at least one document.
Postconditions	AI-generated responses are presented based on the document content.
Main Flow	

- I. The user inputs a query in the chat interface.
- II. The system searches the document embeddings for relevant content.
- III. The AI generates a response based on the search result.
- IV. The response is displayed to the user.

Use Case Name	User Authentication
Description	Handles secure user registration and login via Clerk.
Actors	User, Clerk Service
Preconditions	The user must access the application.
Postconditions	The user is authenticated and granted system access.
Main Flow	

- I. The user enters login credentials or chooses social login.
- II. Clerk service verifies the credentials.
- III. Upon successful authentication, the user is granted access.

Chapter 3: Design Document (For Object Oriented Approach)

3.1. Introduction:

This chapter presents the design approach for the TextTalk application, focusing on the object-oriented design methodology. The design is structured to ensure scalability, maintainability, and effective interaction between the system components.

Following artifacts must be included in the 3rd deliverable.

1. Domain Model
2. System Sequence Diagram
3. Sequence Diagram
4. Collaboration Diagram
5. Operation Contracts
6. Design Class Diagram
7. State Transition Diagram
8. Data Model

Now we discuss these artifacts one by one as follows:

1. Domain Model Prompt:

"Create a Domain Model for the TextTalk project, representing key entities and their relationships. Entities include User, PDF Document, Chat Module, AI Response Generator, and Authentication Service. Show the attributes and associations between these entities."

2. System Sequence Diagram Prompt:

"Develop a System Sequence Diagram for a user uploading a PDF and initiating a chat. Include actors like the User and system components like the PDF Parser, Document Vectorizer, AI Query Processor, and Response Generator."

3. Sequence Diagram Prompt:

"Create a Sequence Diagram for a typical interaction in TextTalk, where a user logs in, uploads a PDF, asks a query, and receives an AI response. Show message flow between the user interface, authentication system, vector database, and AI model."

4. Collaboration Diagram Prompt:

"Draw a Collaboration Diagram showing interactions between the User, PDF Parsing Module, LangChain-based AI Processor, and Response Display Component in the TextTalk system."

5. Operation Contracts Example:

Operation	Description	Preconditions	Postconditions
UploadPDF	Upload a PDF file to the system	User logged in	PDF parsed, vectorized, and stored
GenerateAIResponse	Provide AI-based responses to user queries	PDF already uploaded and parsed	AI response generated and displayed

6. Design Class Diagram Prompt:

"Design a Class Diagram for TextTalk, including classes for User, PDF Document, Chat Session, AI Processor, and Authentication Module. Show their attributes, methods, and relationships."

7. State Transition Diagram Prompt:

"Create a State Transition Diagram for the TextTalk system, showing states such as Logged Out, Logged In, Uploading PDF, Parsing PDF, Querying AI, and Receiving Response."

8. Data Model Prompt:

"Design a Data Model for the TextTalk project, including tables for Users, PDF Documents, Embeddings, and Chat History. Show primary and foreign keys, as well as key attributes for each table."

3.2. Domain Model

Domain model of our website is as follow:

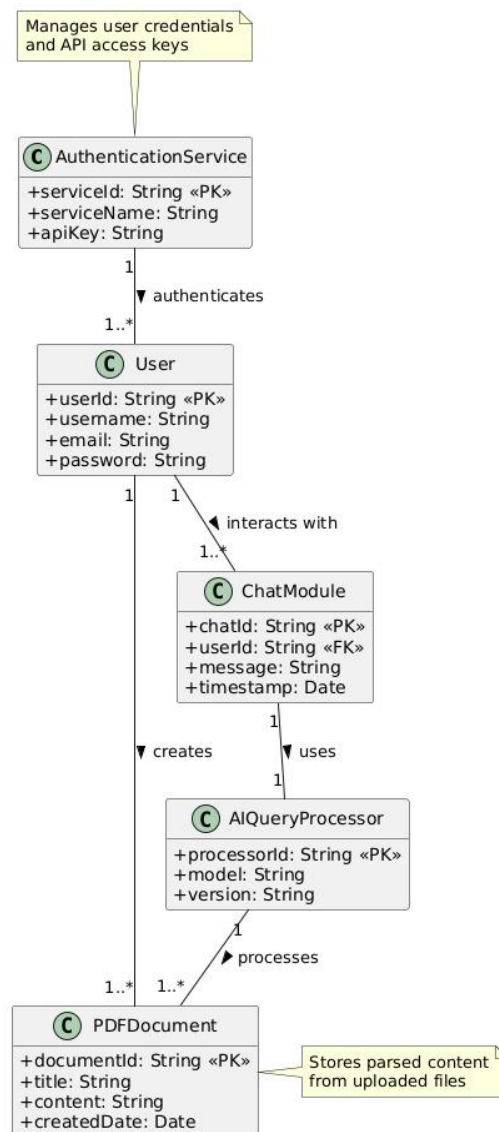


Fig 3.1: Domain model

3.3. System Sequence Diagram

A System Sequence Diagram (SSD) visually represents the sequence of interactions between external actors (users or systems) and the system during specific use cases. It shows the flow of messages, events, and responses, helping to understand system behavior and communication patterns in a structured manner.

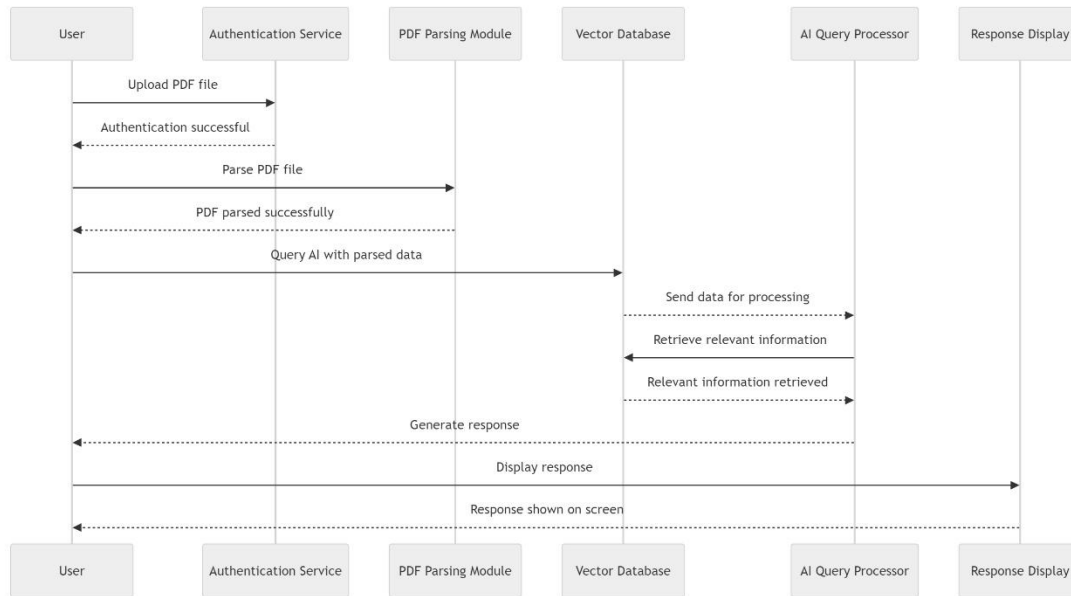
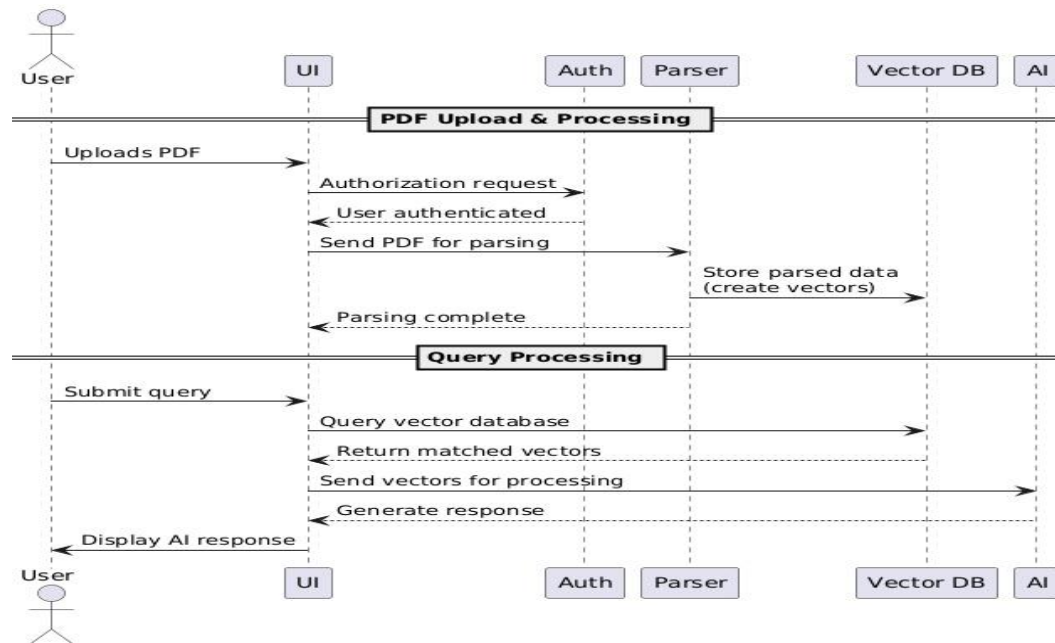


Fig 3.2: System sequence diagram

3.5. Collaboration Diagram



3.6. Operation Contracts

1. Upload PDF

Attribute	Details
Operation Name	Upload PDF
Responsibilities	Uploads and stores PDF file, parses content
Cross Reference	PDF Module
Exceptions	File format not supported, network errors
Preconditions	User is authenticated
Postconditions	PDF successfully uploaded and parsed

2. Authenticate User

Attribute	Details
Operation Name	Authenticate User
Responsibilities	Validates user credentials and grants access
Cross Reference	User Authentication Module
Exceptions	Invalid login credentials, session timeout
Preconditions	User provides valid credentials
Postconditions	User successfully authenticated

3. Generate AI Response

Attribute	Details
Operation Name	AI Response
Responsibilities	Fetches relevant content, generates AI response
Cross Reference	AI Response Module
Exceptions	Query format invalid, server timeout
Preconditions	Document uploaded, valid query received
Postconditions	AI response successfully generated

4. Extract Key Info

Attribute	Details
Operation Name	Extract Key Info
Responsibilities	Extracts key points from uploaded document
Cross Reference	Parsing Module
Exceptions	Corrupted document, inconsistent data
Preconditions	PDF content successfully parsed
Postconditions	Key points extracted and displayed

5. Save Chat History

Attribute	Details
Operation Name	Save Chat History
Responsibilities	Stores user interactions for reference
Cross Reference	Database Module
Exceptions	Database connection failure
Preconditions	User authenticated, valid session
Postconditions	Chat history saved securely

6. Logout User

Attribute	Details
Operation Name	Logout User
Responsibilities	Terminates user session securely
Cross Reference	Session Management
Exceptions	Session not found
Preconditions	User is logged in
Postconditions	User successfully logged out

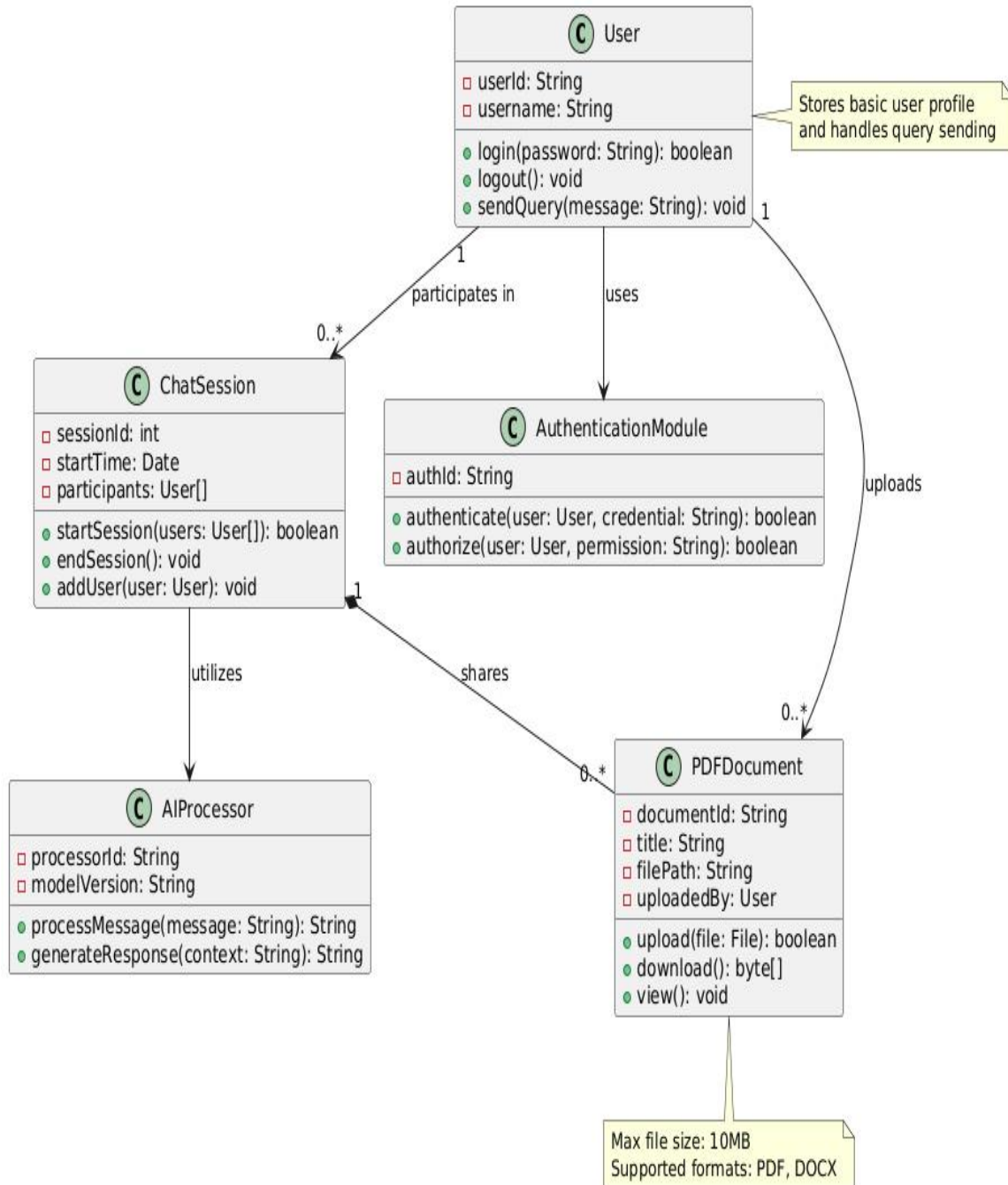
7. Translate Query

Attribute	Details
Operation Name	Translate Query
Responsibilities	Translates query for multilingual support
Cross Reference	Translation Module
Exceptions	Unsupported language error
Preconditions	Query submitted in non-default language
Postconditions	Query translated successfully

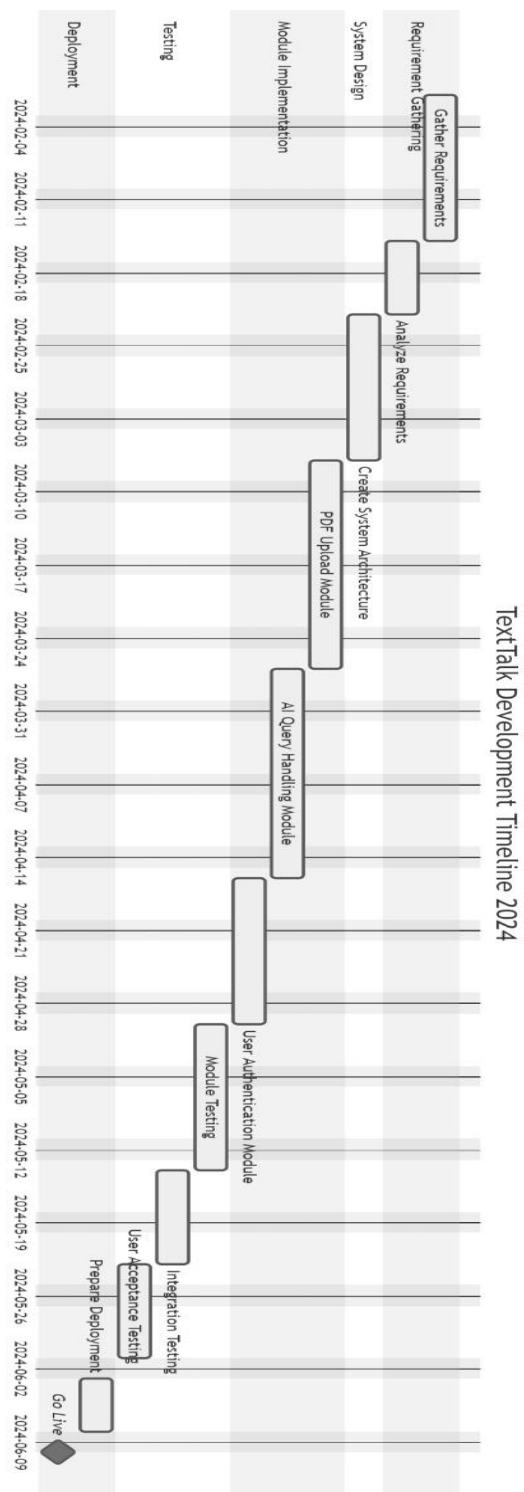
8. Analyze Document

Attribute	Details
Operation Name	Analyze Document
Responsibilities	Performs document analysis
Cross Reference	Document Analysis Module
Exceptions	No related content found
Preconditions	Document is uploaded and parsed
Postconditions	Analysis results displayed

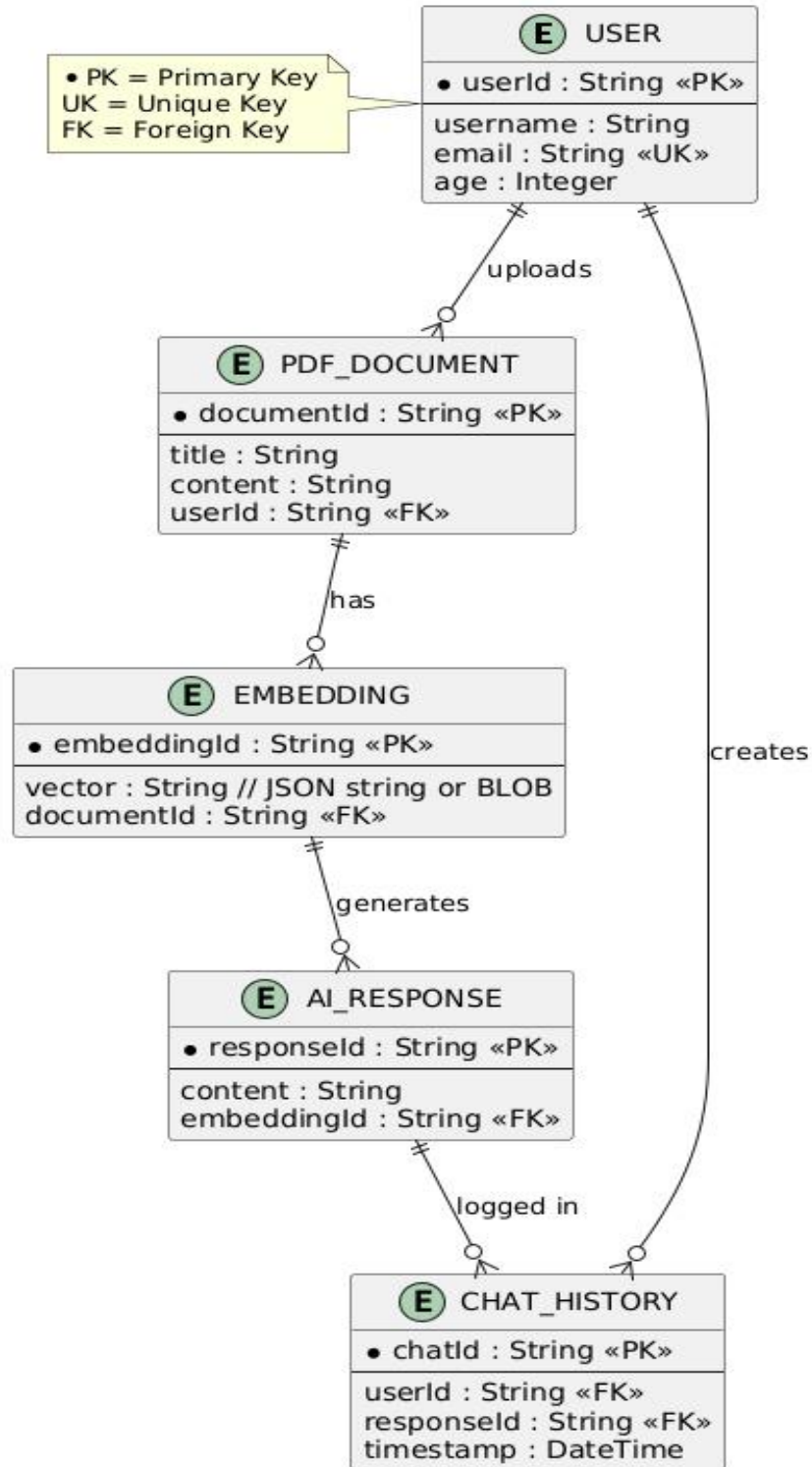
3.7. Design Class Diagram



3.8. Chart diagram:



3.9. Data Model



Chapter 3: Software Requirement Specification and Design Document (For structured Approach)

3.1. Introduction:

This chapter explains the structured design and analysis approach used in the development of **TextTalk**, a web-based AI application that allows users to interact with PDF documents. The structured methodology focuses on data flow, behavior modeling, and component-based design. Below are the main models created for this approach:

1. Entity Relationship Diagram
2. Data Flow Diagram (Functional Model)
3. State Transition Diagram (Behavioral Model)
4. Architecture Design
5. Component Level Design

3.2. Entity Relationship Diagram:

The ERD describes the logical structure of the application's database.

Entities and Relationships:

- **User** (UserID, Name, Email, Password)
- **Document** (DocumentID, UserID, FileName, UploadDate)
- **Query** (QueryID, DocumentID, UserID, QueryText, Timestamp)
- **Response** (ResponseID, QueryID, AIText, Timestamp)
- **History** (UserID, QueryID, DocumentID)

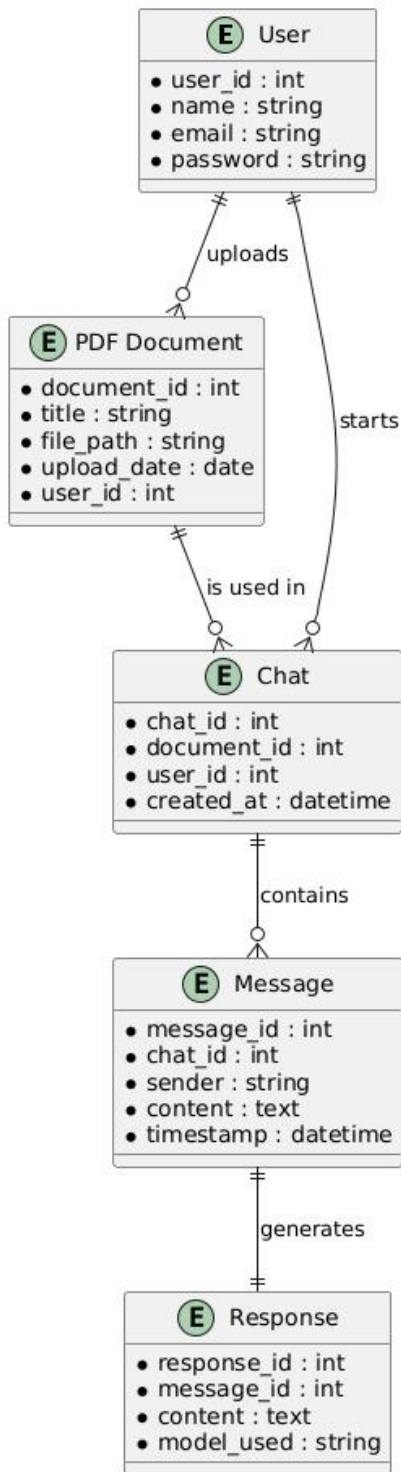
Relationships:

- A User can upload many Documents
- A Document can have many Queries
- A Query returns one AI Response

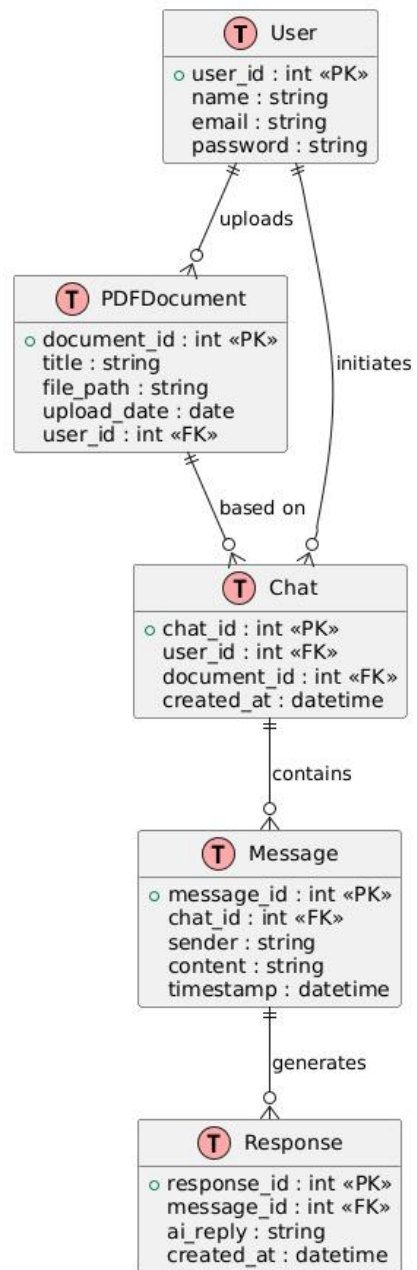
ERD:

Entity Relationship Diagram is on next page.

ERD:



Fully Dressed ERD:



3.3. Data flow diagram (Functional Model)

Context-Level DFD:

External Entities:

- User
- Clerk Authentication
- LLM (GPT/Gemini)

Processes:

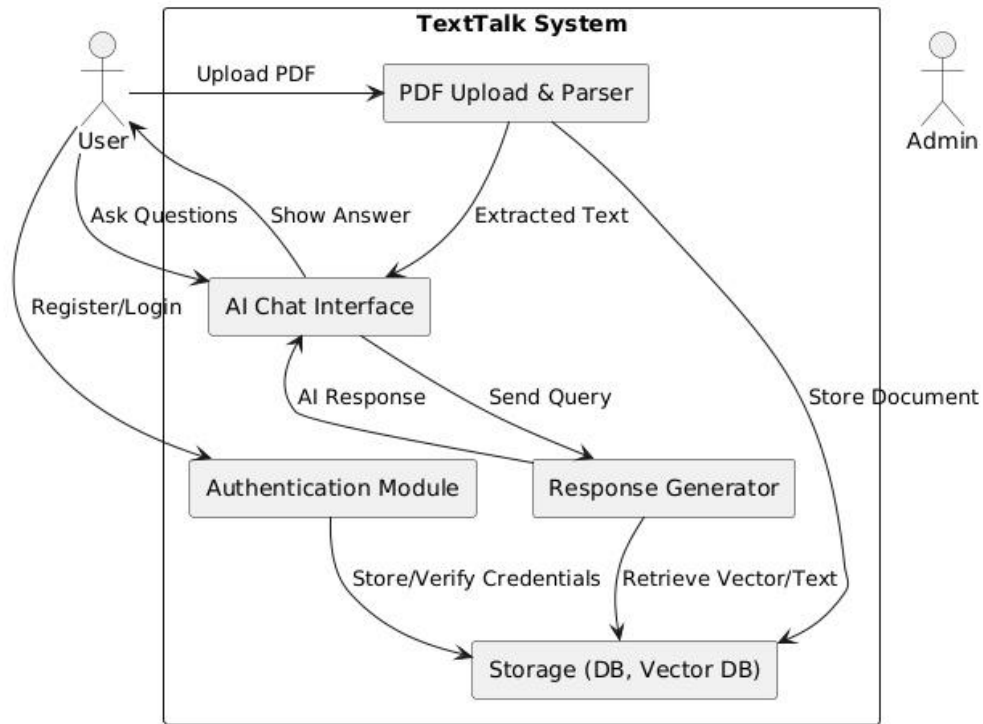
- Login
- Upload PDF
- Parse and Embed
- Ask Query
- Get Response

Level-1 DFD:

1. **Authenticate User:** Login/register using Clerk
2. **Upload Document:** Validate and store PDF
3. **Vectorize PDF:** Convert text to embeddings
4. **Chat with Document:** Send queries and receive answers
5. **Save Chat History:** Store user queries and AI replies

DFD:

Image is on next page.



3.4. State Transition Diagram

Shows how the system moves between different states based on user actions.

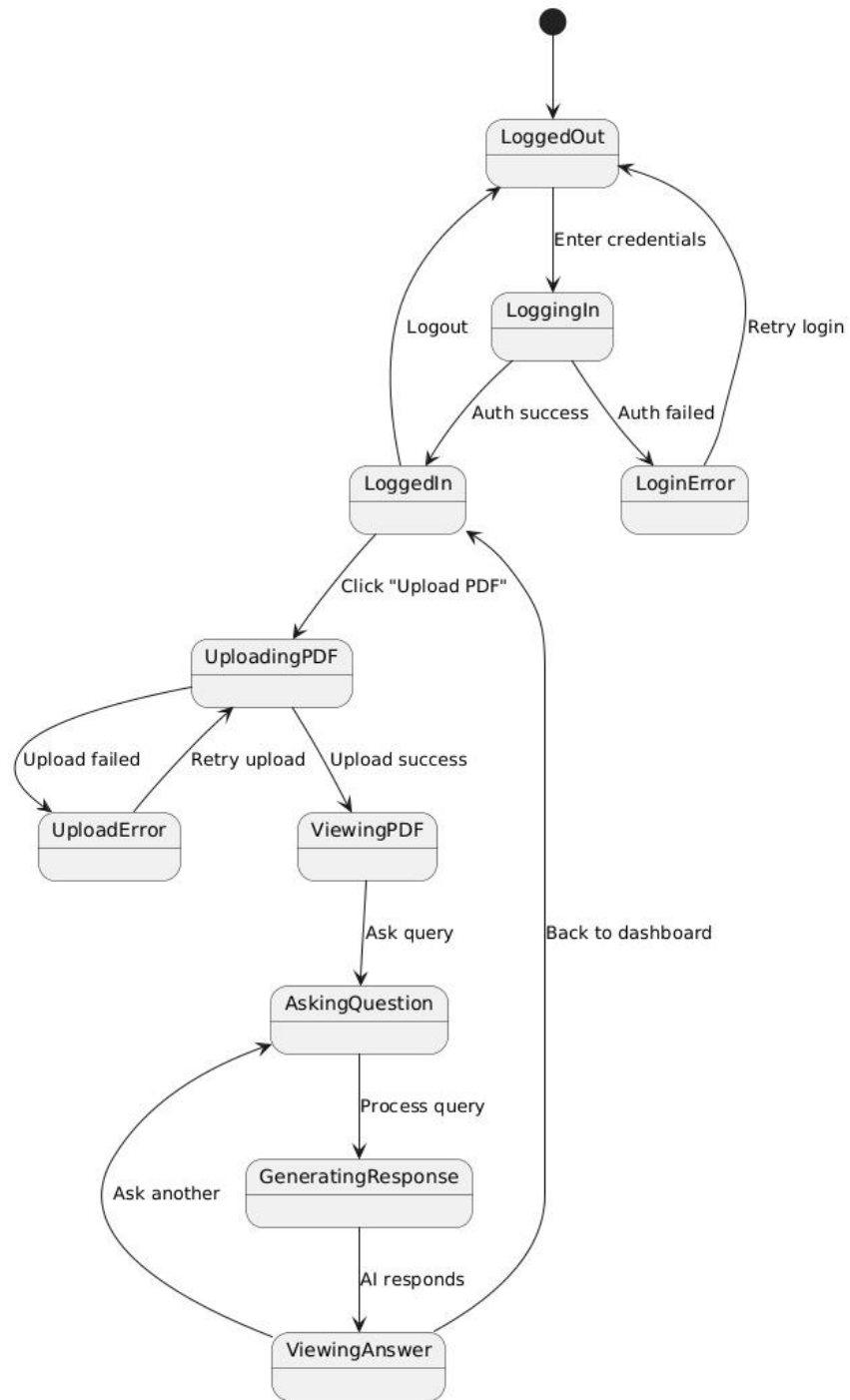
States:

- Logged Out
- Logged In
- Uploading Document
- Document Parsed
- Query Sent
- Response Displayed
- Session Ended

Transitions:

- After login → Logged In
- Upload complete → Document Parsed
- Query asked → Query Sent → Response Displayed

STD:



3.5. Architectural design

TextTalk uses a modular client-server web architecture:

Frontend (Client-Side):

- Built in Next.js
- Tailwind CSS for styling
- Handles routing, upload interface, and chat UI

Backend (Server-Side):

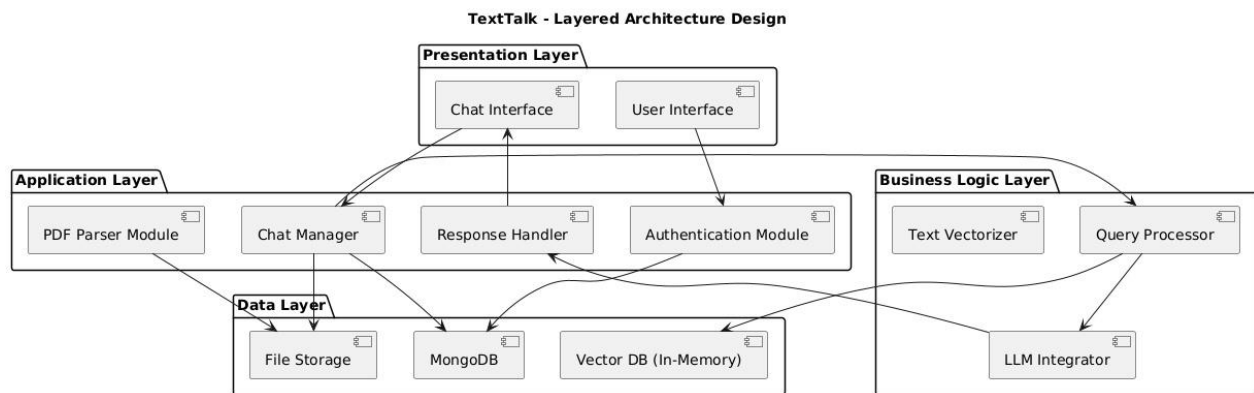
- Node.js & LangChain.js for parsing, vectorization, and AI interaction
- Clerk integration for user authentication

Databases:

- MongoDB (User and Document data)
- In-Memory Vector Database (Embeddings)

Deployment:

- Frontend hosted on Vercel
- Backend APIs deployed to cloud



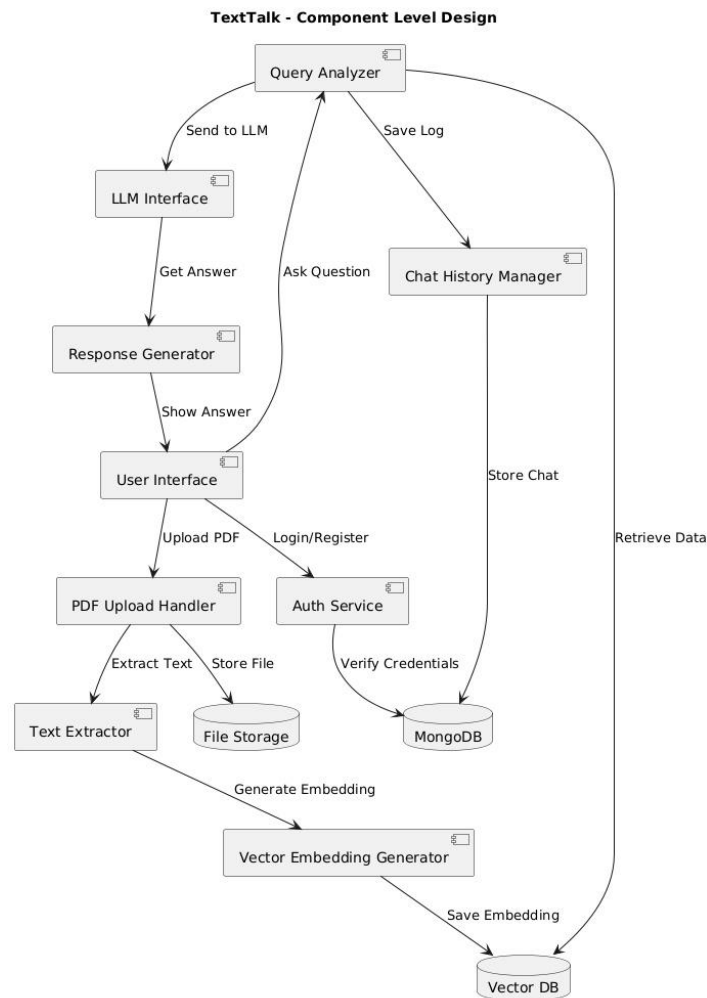
3.6. Component Level Design

This level breaks the system into core components:

1. **Authentication Component:** Manages login/register using Clerk
2. **Upload Component:** Allows PDF drag-and-drop upload
3. **Parser Component:** Extracts text from uploaded PDF
4. **Vectorizer Component:** Converts document text into embeddings
5. **Chat Component:** Sends user queries to AI and displays results
6. **Response Handler:** Links response to original PDF section
7. **History Component:** Displays past queries and responses
8. **Multilingual Module:** Translates input/output using selected language

These models ensure that *TextTalk* is well-structured, efficient, and scalable for future enhancements.

CLD:



Chapter 4: User Interface Design

4.1. Introduction

The User Interface (UI) design of **TextTalk** focuses on providing a simple, user-friendly, and responsive experience. The main goal is to allow users to interact with PDF documents using natural language in a clean and interactive environment. The design is created using **Next.js** and **Tailwind CSS**, which provide a modern, fast, and responsive interface for all devices.

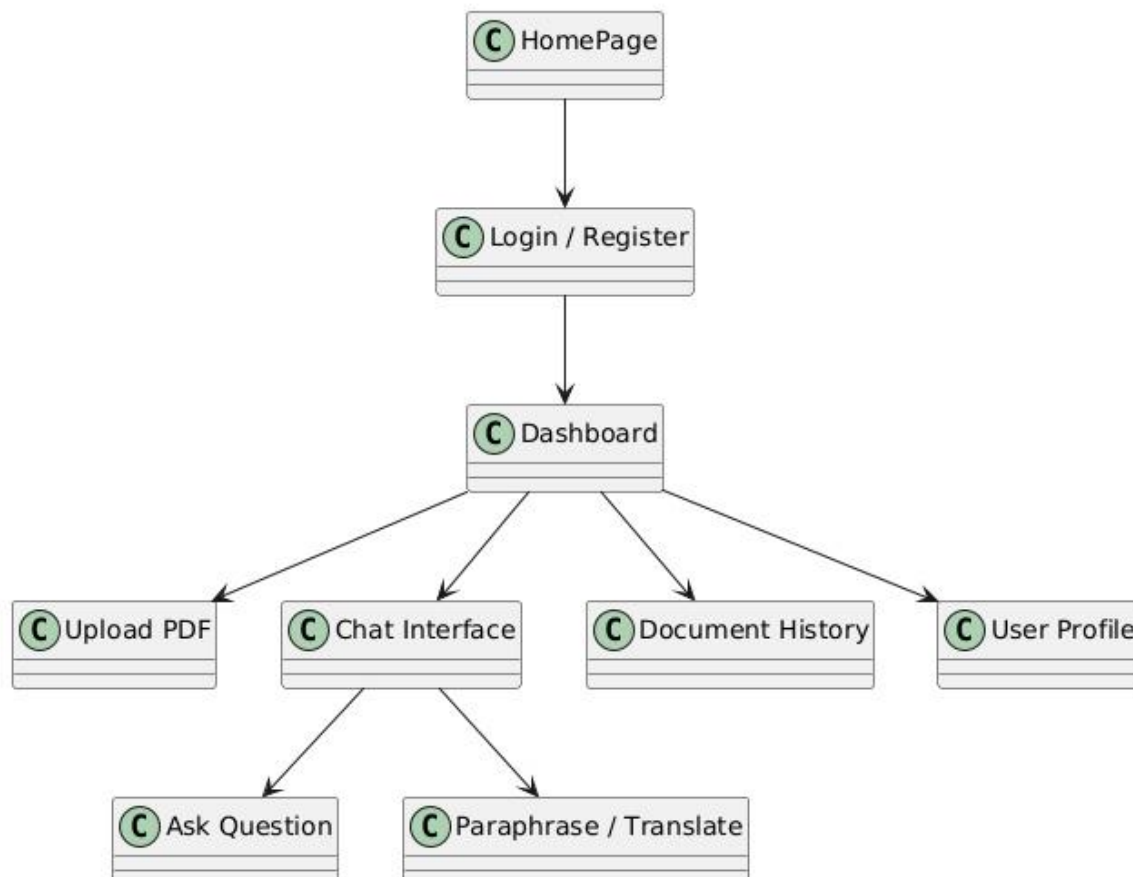
The UI is also designed with accessibility and mobile responsiveness in mind, ensuring a consistent and user-friendly experience across all screen sizes.

4.2. Site Maps

The site map defines the main structure of the web application and shows how different pages are connected. Below is the basic structure:

- Home Page
 - └─ Login / Register
 - └─ Dashboard
 - └─ Upload PDF
 - └─ Chat Interface
 - └─ Ask Question
 - └─ Paraphrase / Translate
 - └─ Document History
 - └─ User Profile

Site Map - TextTalk Application

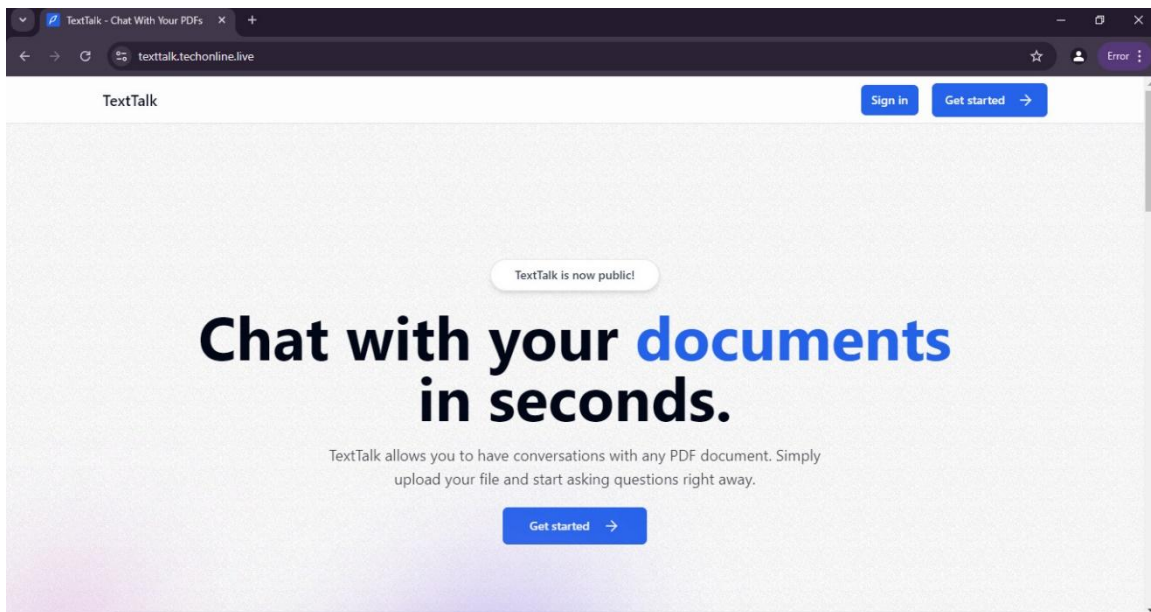


4.3. Story boards

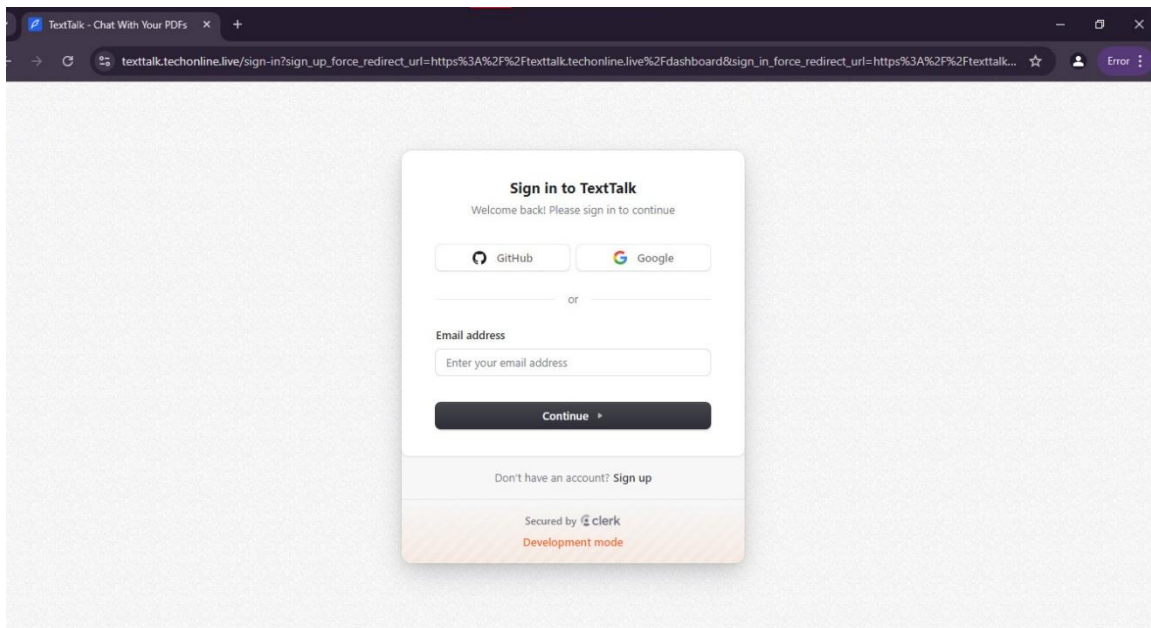
Storyboard 1: User Login & Registration

Step	Description
1	User visits the Home page
2	User clicks on “Login” or “Register”
3	Clerk handles authentication (social login or email)
4	On success, user is redirected to dashboard

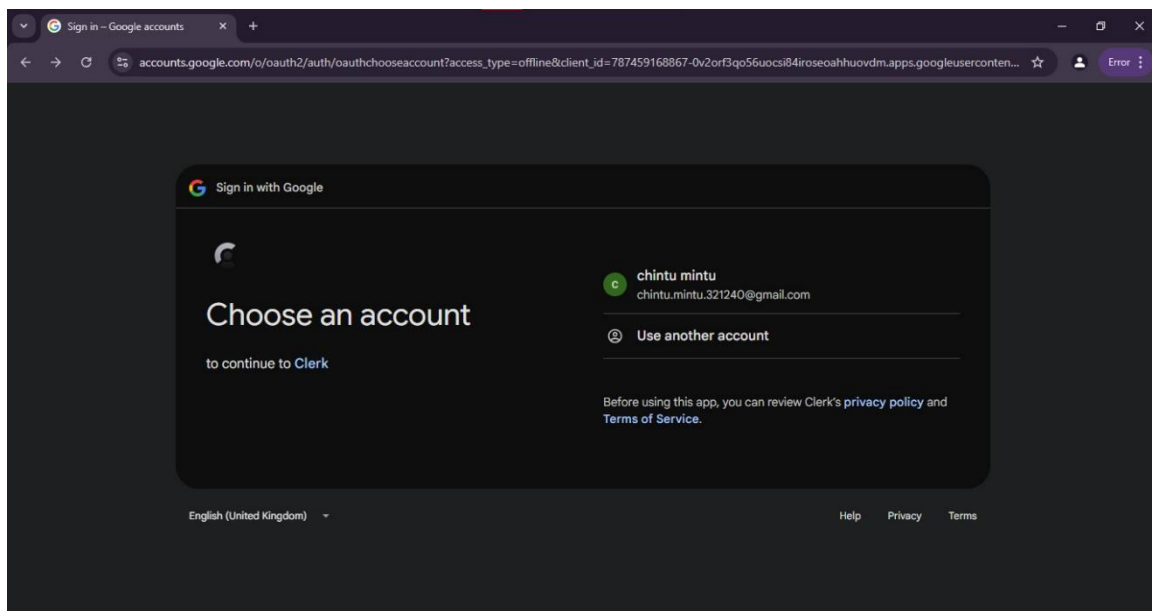
Home Page:

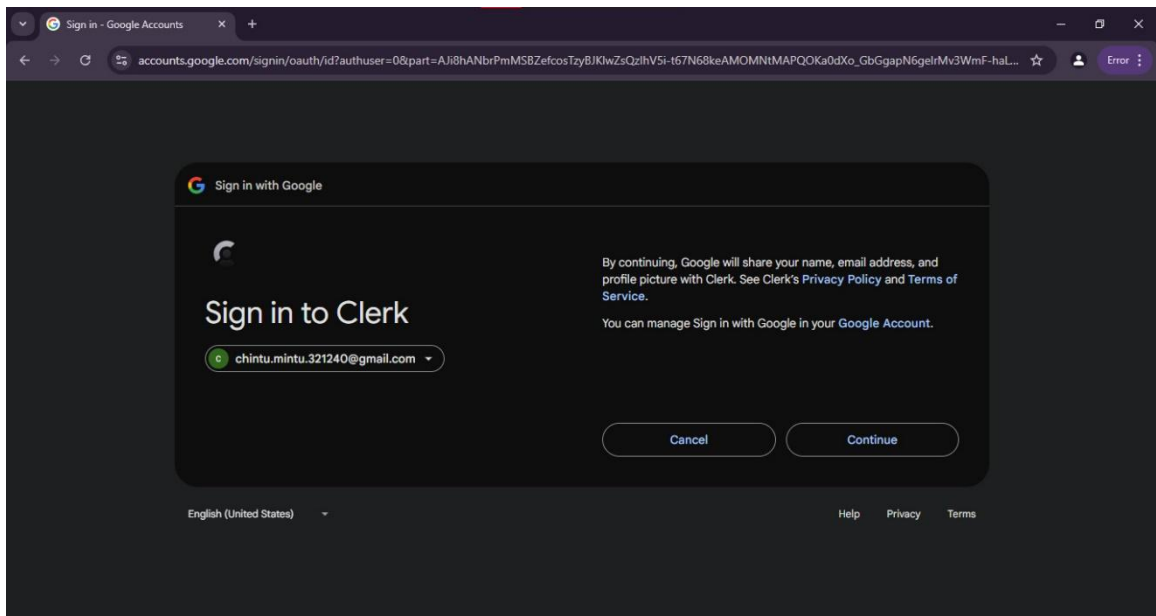


Login or Register:

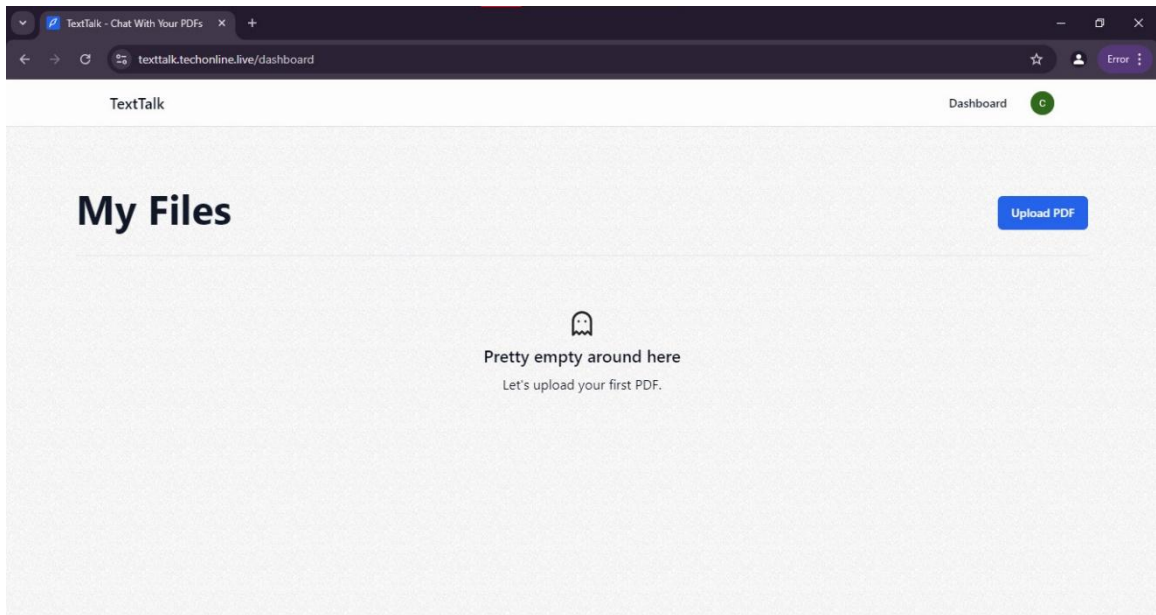


Clear Handles Authentication:





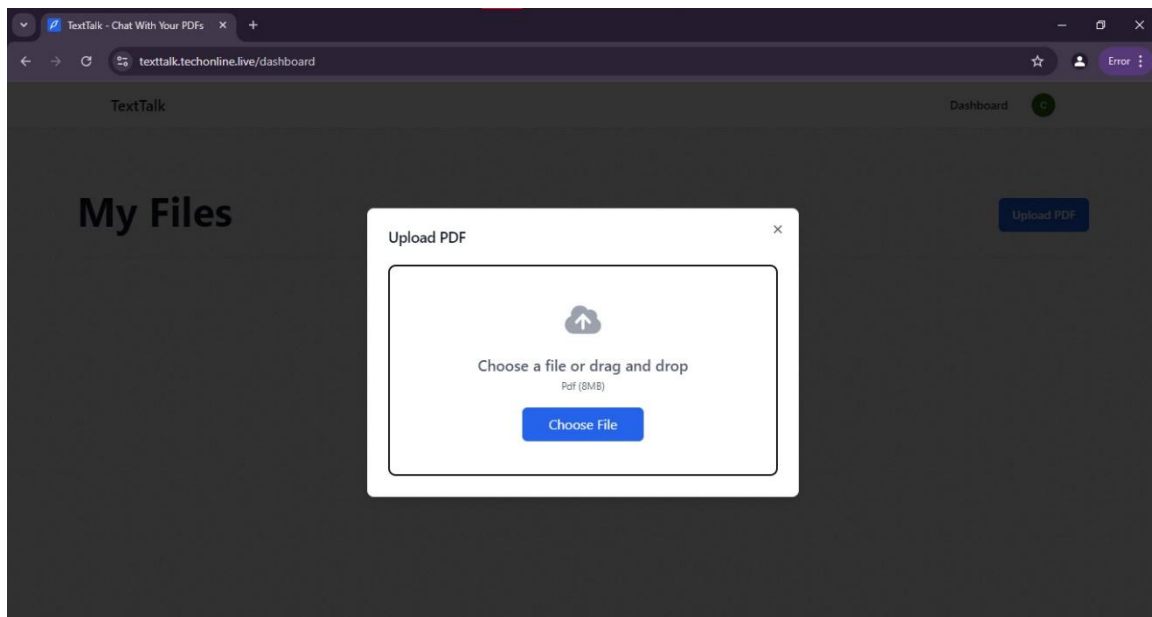
On Success:



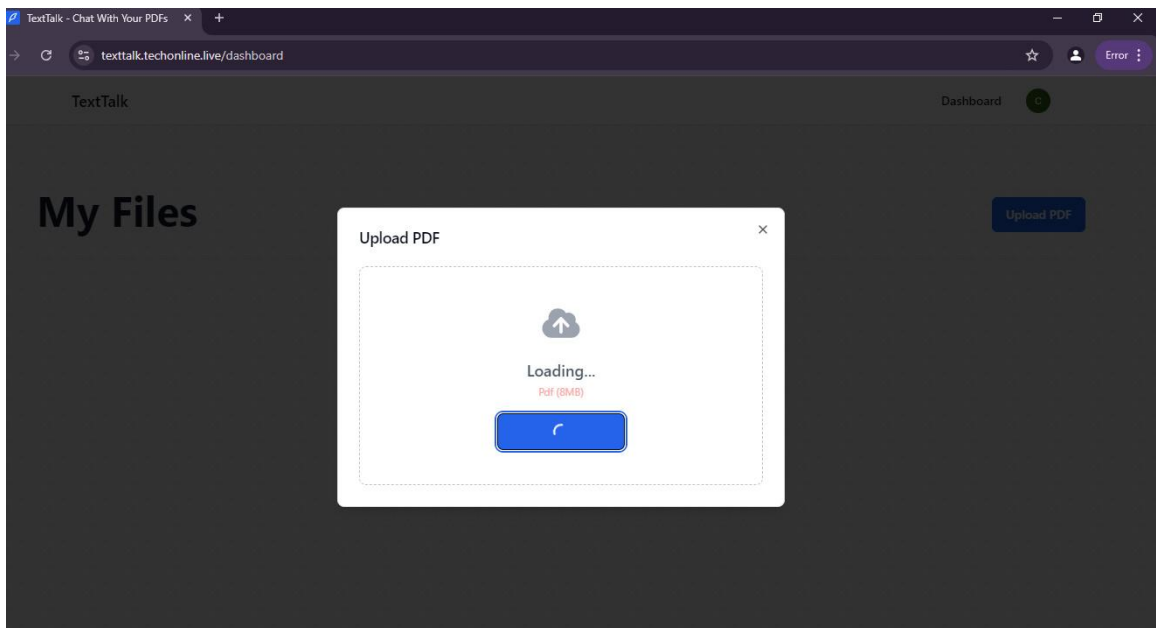
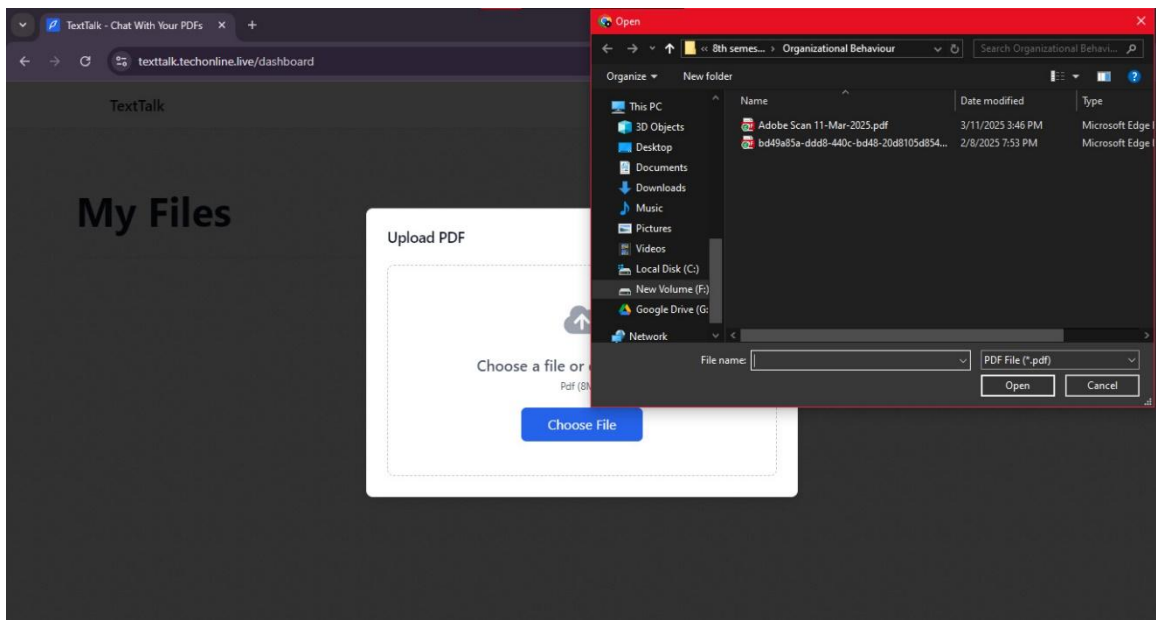
Storyboard 2: Uploading and Chatting with PDF

Step	Description
1	User clicks “Upload PDF”
2	User selects a PDF from their device
3	System extracts content and creates embeddings
4	User enters query in chat box
5	AI generates response using LangChain + LLM
6	User views and interacts with the response

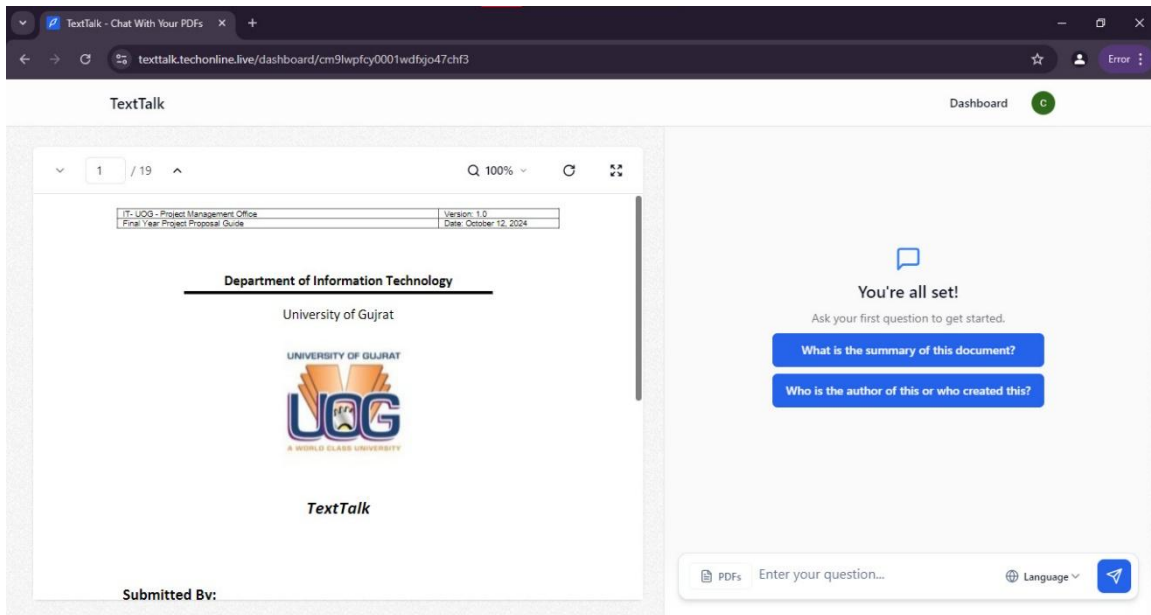
Upload PDF:



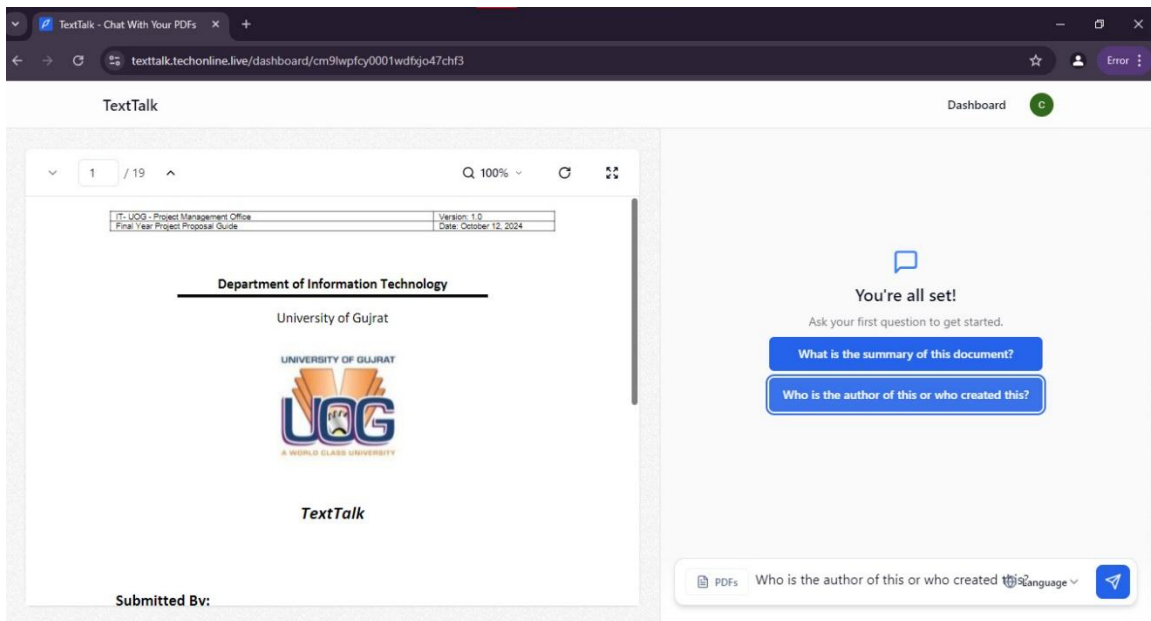
Select PDF from the Device:



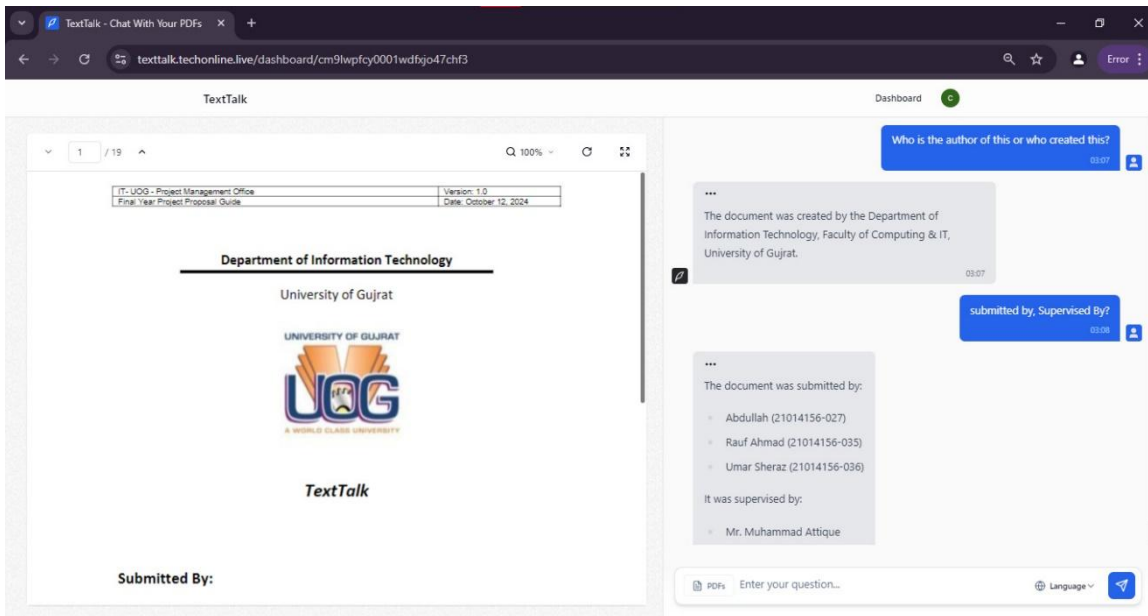
System extracts content and creates embeddings:



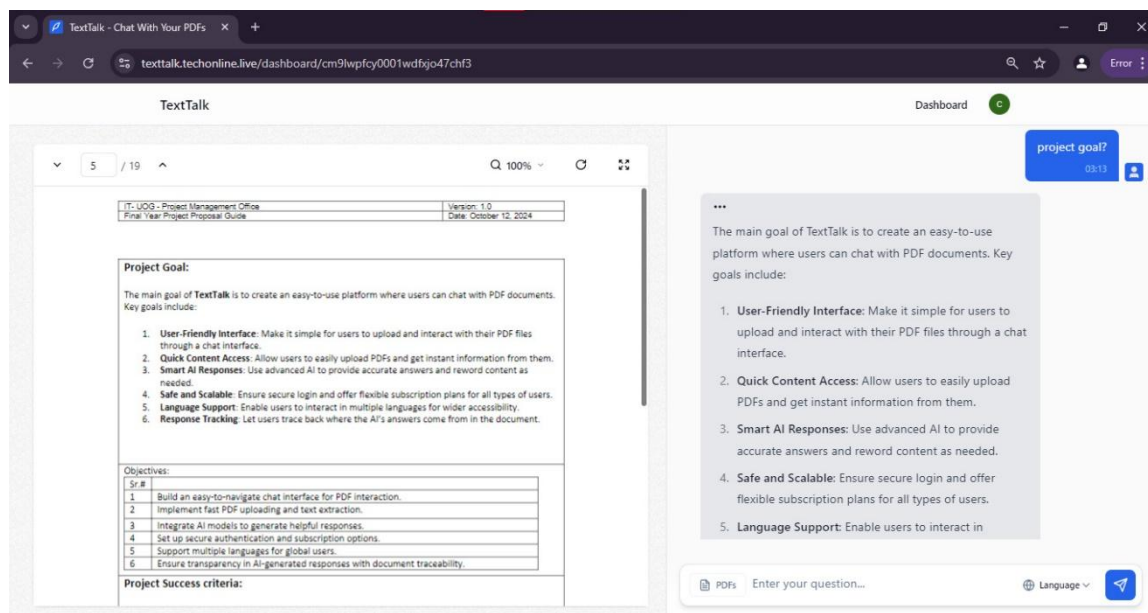
User enters query in chat box:



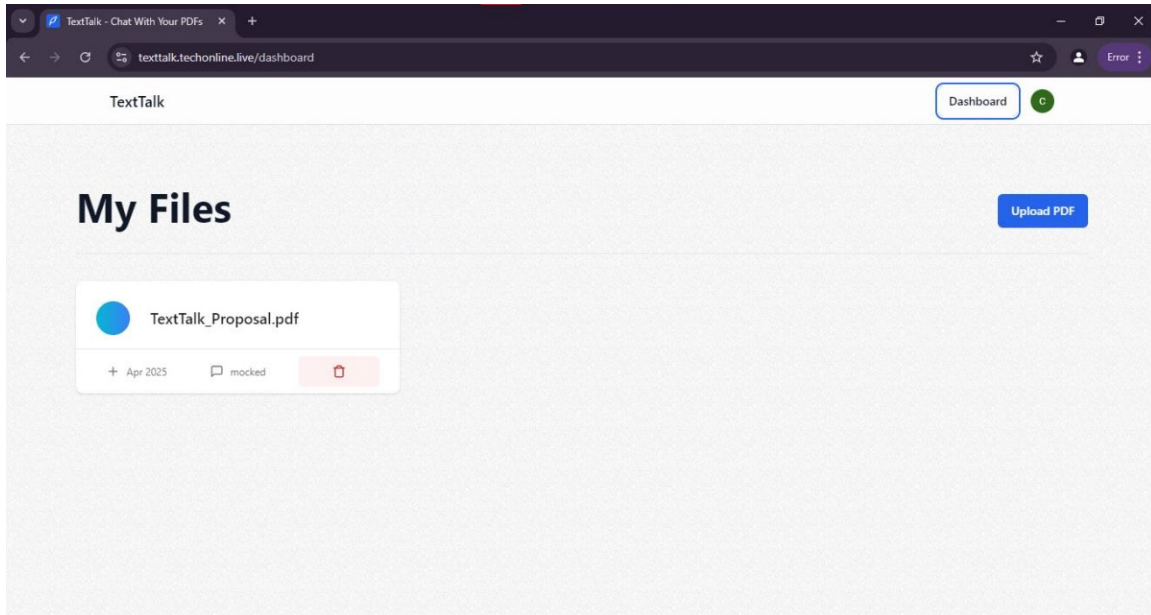
AI generates response using LangChain + LLM:



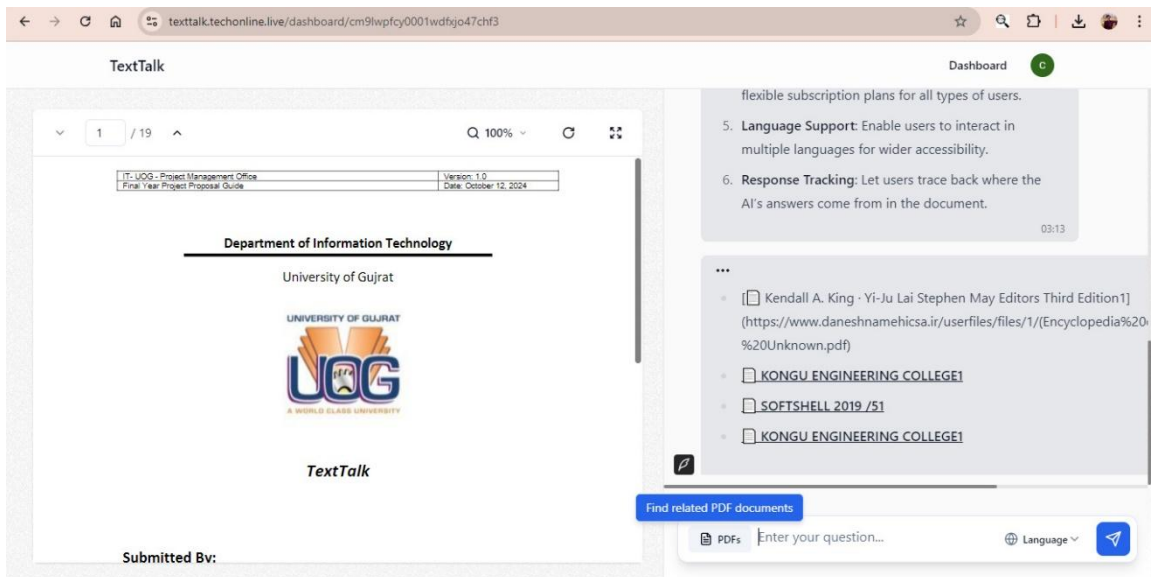
User views and interacts with the response:



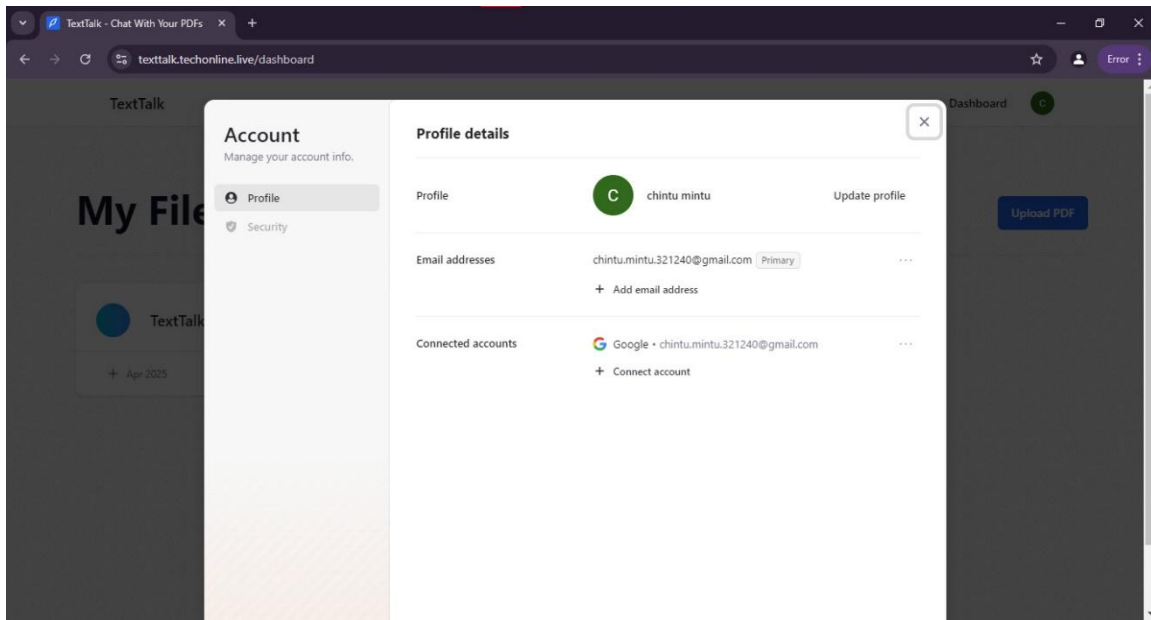
Dashboard after Interacting:



Find Related PDFs:

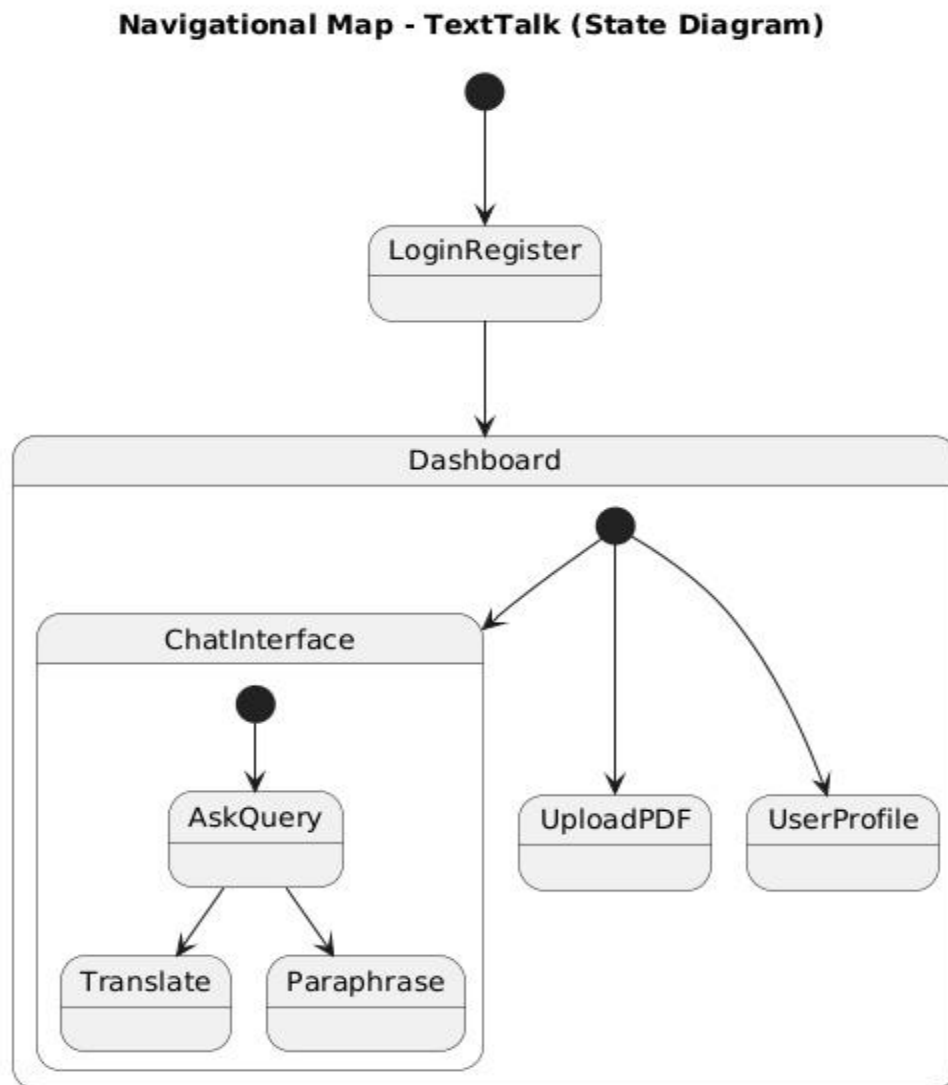


Account Management:



4.4. Navigational maps:

This shows how users move between screens/modules:



4.5 Trace-ability Matrix

This matrix maps requirements (from SRS) to design and implementation components:

Requirement ID	Requirement Description	UI Component	Module / Feature
RQ-01	User login and authentication	Login/Register Page	Clerk Authentication Module
RQ-02	Upload PDF document	Upload Button / File Input	PDF Upload & Parser Module
RQ-03	Store uploaded PDFs in secure storage	N/A	File Storage (e.g., Mongo/GridFS)
RQ-04	Ask questions from uploaded PDF	Chat Interface	LangChain + LLM Chat Module
RQ-05	Responsive and mobile-friendly UI	All Screens	Tailwind CSS + Next.js
RQ-06	Generate meaningful AI responses	Chat Output Display	LLM Response Generator
RQ-07	Paraphrase PDF content	Paraphrase Button	Paraphrasing Module
RQ-08	Translate content into multiple languages	Language Selector	Translation API Module
RQ-09	View AI response source inside document	“Source” link under response	Traceability Highlight Module
RQ-10	View history of chat interactions	Document History Section	Chat History Manager
RQ-11	Ensure secure and authorized data access	Auth + Dashboard Routing	Clerk Auth + Session Handler
RQ-12	Efficient query processing and vector matching	N/A	Vector DB + Embedding Engine
RQ-13	Scalable backend for concurrent users	N/A	Next.js Server Functions

Chapter 5: Software Testing

5.1 Introduction:

This deliverable follows the IEEE Standard 829-1998 for Software Test Documentation. This standard provides a set of test documents that cover dynamic software testing (i.e., running the software and executing code). It outlines documentation that can be used across various types of software (e.g., academic, commercial, scientific) regardless of size, complexity, or criticality.

It includes initial development testing and testing for future software versions. The documentation is flexible and allows organizations to decide which test phases and documents are necessary. The standard does not specify testing methods or tools, but provides a structure for managing testing activities and reporting outcomes.

Standard Artifacts in this Chapter:

1. Test Plan
2. Test Design Specification
3. Test Case Specification
4. Test Procedure Specification
5. Test Item Transmittal Report
6. Test Log
7. Test Incident Report
8. Test Summary Report

5.2. Test plan

5.2.1 Purpose

The purpose of this test plan is to define the scope, approach, resources, and schedule for testing the TextTalk web application.

5.2.2 Test Plan Outline

- a) **Test Plan Identifier:** TP-TEXTTALK-01
- b) **Introduction:** TextTalk is a web-based AI-powered document interaction tool. Testing will cover login, document upload, AI chat interaction, paraphrasing, and multi-language features.
- c) **Test Items:**
 - User Authentication Module
 - PDF Upload and Parsing Module
 - AI Chat Module
 - Paraphrasing & Rewriting Module
 - Traceability & Verification Module
- d) **Features to Be Tested:**

- Secure login and authentication
- Uploading and parsing PDFs
- Chat interaction with PDF content
- AI-based paraphrasing
- Multi-language support

e) **Features Not to Be Tested:**

- Offline mode
- Mobile app compatibility
- Video/audio processing

f) **Approach:**

- Functional testing using manual and automated methods
- Unit testing with Jest (JavaScript)
- Integration testing of modules
- User acceptance testing (UAT)

g) **Item Pass/Fail Criteria:**

- Test cases must return expected results
- No critical bugs or crashes

h) **Suspension Criteria & Resumption:**

- Suspend if a critical bug blocks further testing
- Resume after bug is fixed and verified

i) **Test Deliverables:**

- Test plan document
- Test case documents
- Test execution logs
- Bug reports
- Final test summary

j) **Testing Tasks:**

- Design test cases
- Setup environment
- Execute tests
- Record results

k) **Environmental Needs:**

- Chrome browser (latest)
- Windows/macOS OS

- Internet connection

l) **Responsibilities:**

- Testers: Rauf Ahmad & Umar Sheraz
- Supervisor: Mr. Muhammad Attique

m) **Staffing & Training Needs:**

- Basic knowledge of AI chat tools
- Familiarity with web testing (React, Tailwind, MongoDB)

n) **Schedule:**

- Start Date: 1st June 2025
- End Date: 30th June 2025

o) **Risks & Contingencies:**

- Delay in AI integration -> Use fallback AI model
- Server downtime -> Extend testing hours

p) **Approvals:**

- Project Supervisor: Mr. Muhammad Attique
- QA Reviewer: To be assigned

5.3. Test design specification

5.3.1 Purpose

To define the test approach and design for the main features in TextTalk.

5.3.2 Outline

- a) **Test Plan Identifier:** TDS-TEXTTALK-01
- b) **Introduction:** Testing includes document interaction features such as chat, paraphrasing, and response tracing. Related documents: SRS, Design Spec.
- c) **c. Test Items:**
 - PDF Upload Component
 - Chat Interface
 - AI Response Generator
- d) **Features to Be Tested:**

- Upload validation
- Response relevance
- Paraphrase accuracy

e) **Features Not to Be Tested:**

- Advanced analytics (planned for future)

f) **Approach:**

- Black box testing
- Manual functional test execution
- Input validation and output accuracy

g) **Item Pass/Fail Criteria:**

- Feature behaves as expected without errors

h) **Suspension Criteria:**

- System crash or failed module

i) **Deliverables:**

- Test Plan
- Test Cases
- Execution Logs

j) **Testing Tasks:**

- Design test cases
- Execute tests
- File bugs

k) **Environmental Needs:**

- Google Chrome, Internet

l) **Responsibilities:**

- Test Execution: Rauf Ahmad
- Logging: Umar Sheraz

m) **Staffing Needs:**

- Intermediate testing knowledge

n) **Schedule:** June 2025

o) **Risks:**

- High model latency -> Mock response fallback

p) **Approvals:**

- QA Team Lead

5.4. Test Case Specification

5.4.1 Purpose

To define individual test cases for validating system behavior.

5.4.2 Outline

- a) **Identifier:** TC-TEXTTALK-01
- b) **Test Item:** PDF Upload
- c) **Input Specifications:** Select PDF file (.pdf) from system
- d) **Output Specifications:** Success message, visible file name
- e) **Environmental Needs:** Internet, Browser
- f) **Special Requirements:** File size < 20MB
- g) **Dependencies:** User must be logged in

5.5. Test procedure specification

5.5.1 Purpose

Define exact steps for executing each test case.

5.5.2 Outline

- a) **Identifier:** TP-TEXTTALK-UPLOAD-01
- b) **Purpose:** Test successful PDF upload
- c) **Special Requirements:** Logged-in user
- d) **Procedure Steps:**
 - 1. Log entry in test log
 - 2. Open browser and log in
 - 3. Go to Upload page
 - 4. Click 'Upload PDF'
 - 5. Select test document
 - 6. Confirm success message appears
 - 7. Log test result
 - 8. Wrap up test environment

9. Handle any failed states or errors

5.6. Test item transmittal report

5.6.1. Purpose

To identify the modules of *TextTalk* that are being sent for testing. This report mentions their location, version, responsible team members, and any updates or changes made since development.

5.6.2. Outline

a) Transmittal Report Identifier

TITR-TEXTTALK-001

b) Transmitted Items

Module	Version	Responsible	Linked Document
PDF Upload Module	v1.0	Dev Team A	SRS, Design Doc
AI Chat Module	v1.2	Dev Team B	Design Doc
Paraphrasing Feature	v1.0	Dev Team B	SRS, Test Plan

c) Location

Hosted at: <https://texttalk.techonline.live>

Repository: GitHub (Private repo) – available to test team

d) Status

- All items marked "Ready for Testing"
- No known critical bugs
- Minor UI refinements pending

e) Approvals

Name	Role	Signature	Date
Abdullah	QA Reviewer		2025-04-18
Rauf Ahmad	QA Reviewer		2025-04-18
Umar Sheraz	QA Reviewer		2025-04-18
Mr. Muhammad Attique	Project Supervisor		2025-04-18

5.7. Test log

5.7.1. Purpose

To maintain a step-by-step timeline of testing activities performed on *TextTalk* during module testing.

5.7.2. Outline

a) **Test Log Identifier**

TL-TEXTTALK-001

b) **Description**

- Items tested: PDF Upload, Chat Module, Paraphrasing, Translation
- Test Environment: Chrome v121, Windows 11, 8GB RAM
- Server: Hosted on Vercel (Next.js)

c) **Activity and Event Entries**

Date	Time	Action	Result	Notes
2025-04-15	10:00 AM	Upload test PDF	Pass	Fast response
2025-04-15	10:20 AM	Ask query about page 2	Pass	Relevant output
2025-04-15	11:00 AM	Paraphrase text	Pass	Accurate results
2025-04-15	11:30 AM	Upload corrupt file	Fail	Proper error shown

5.8. Test incident report

5.8.1. Purpose

To report any unexpected bugs, errors, or behavior during testing of *TextTalk*.

5.8.2. Outline

A test incident report shall have the following structure:

- a. Test incident report identifier
- b. Summary
- c. Incident description
- d. Impact

a) **Test Incident Report Identifier**

TIR-TEXTTALK-001

b) **Summary**

Issue with uploading encrypted PDF. App did not show a friendly message.

c) **Incident Description**

- **Input:** Encrypted sample.pdf
- **Expected Result:** Show error "File is locked or unsupported"
- **Actual Result:** App crashed silently
- **Anomaly:** UI freeze
- **Date:** 2025-04-15
- **Test Step:** Upload > Select File
- **Environment:** Chrome 121, Windows 11
- **Testers:** Ali
- **Observers:** QA Lead
- **Attempts:** 2 (issue repeated)

d) **Impact**

Must fix before deployment. Affects PDF Upload Module and user experience.

5.9. Test summary report

5.9.1. Purpose

To summarize all testing done on *TextTalk*, covering what worked, what didn't, and whether the app meets the criteria for delivery.

5.9.2. Outline

A test summary report shall have the following structure:

- a. Test summary report identifier
- b. Summary
- c. Variances
- d. Comprehensive assessment
- e. Summary of results
- f. Evaluation
- g. Summary of activities
- h. Approvals

a) **Summary Report Identifier**

TSR-TEXTTALK-001

b) **Summary**

Modules tested: Login, Upload, Chat, Paraphrasing, Translation

Environment: Chrome, Vercel-hosted app

References: Test Plan, Test Logs, Incident Reports

c) **Variances**

- Only minor UI changes from design

- One failed case (encrypted PDF) found

d) **Comprehensiveness Assessment**

all major features tested. Limited time for browser compatibility testing.

e) **Summary of Results**

Module	Status
Login & Auth	✓ Pass
PDF Upload	⚠ Minor Issue
AI Chat	✓ Pass
Paraphrasing	✓ Pass
Multi-language	✓ Pass

f) **Evaluation**

System is stable and functional. One non-critical issue reported. Ready for deployment after minor fix.

g) **Summary of Activities**

- 3 testers over 2 days
- Approx. 6 hours of active testing
- 1 test case failed
- 9 test cases passed

h) **Approvals**

Name	Role	Signature	Date
Abdullah	QA Lead		2025-04-18
Rauf Ahmad	Tester		2025-04-18
Umar Sheraz	Test Logger		2025-04-18
Mr. Muhammad Attique	Supervisor		2025-04-18

